Quantitative Physiology Clab 3: Simulating Neuronal Models Equations
Max Holmes

Problem 1a

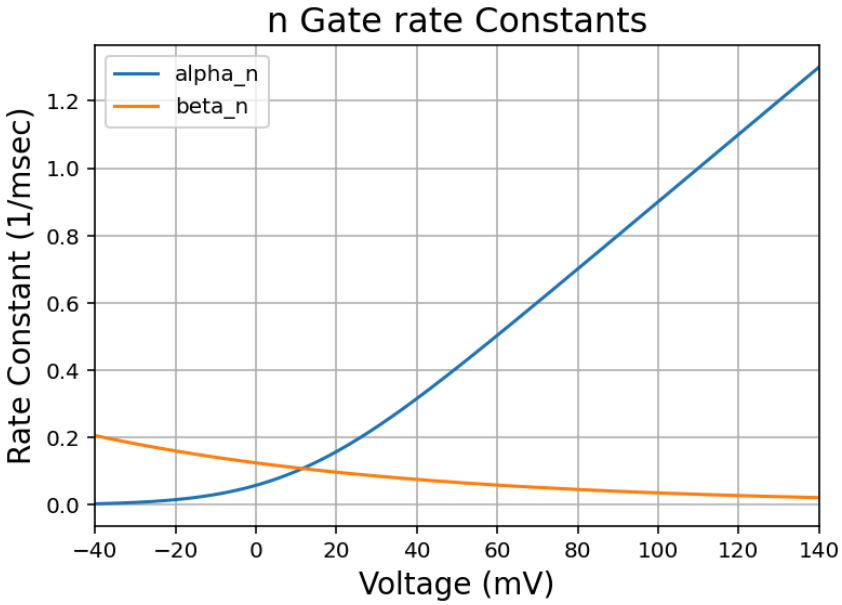

Figure 1 *Rate Constants for Potassium n-gate*

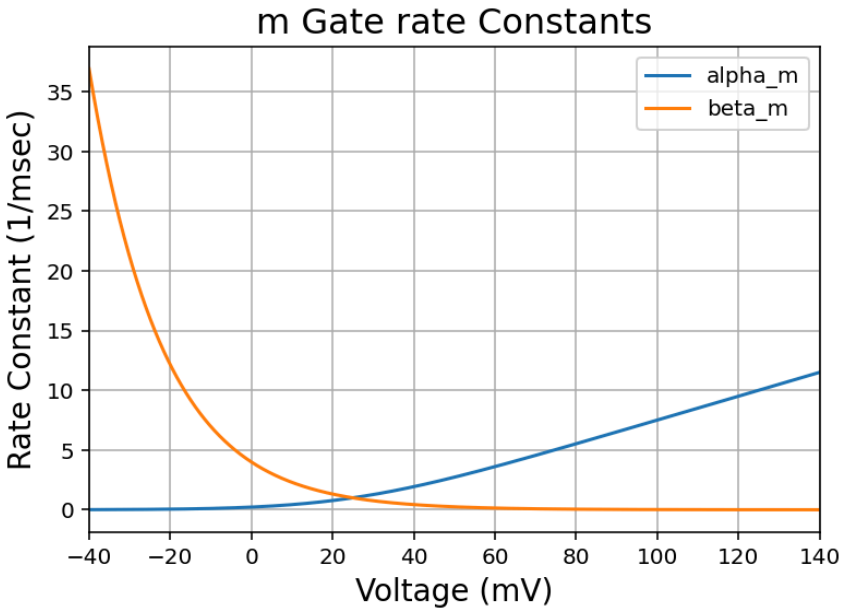

Figure 2 *Rate Constants for Sodium m-gate*

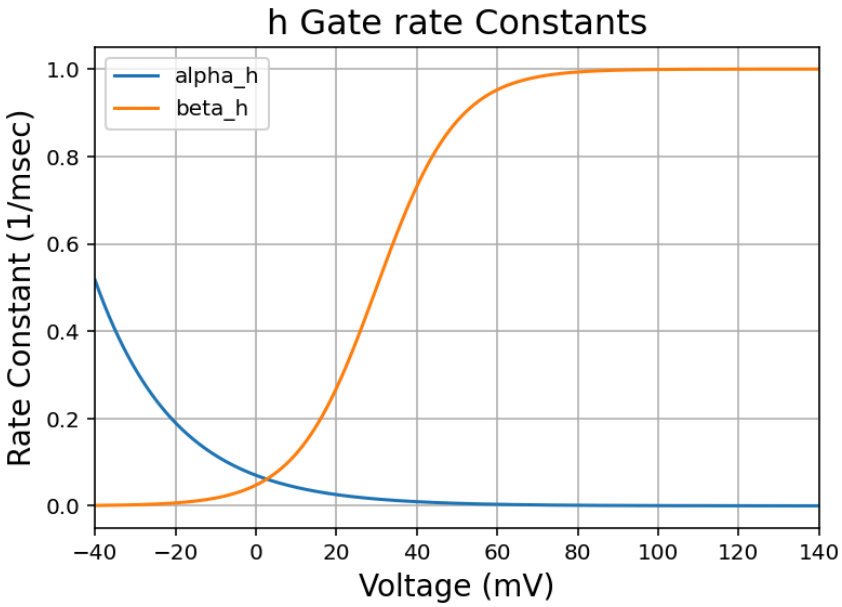

Figure 3 *Rate Constants for Sodium h-gate*

Problem 1b

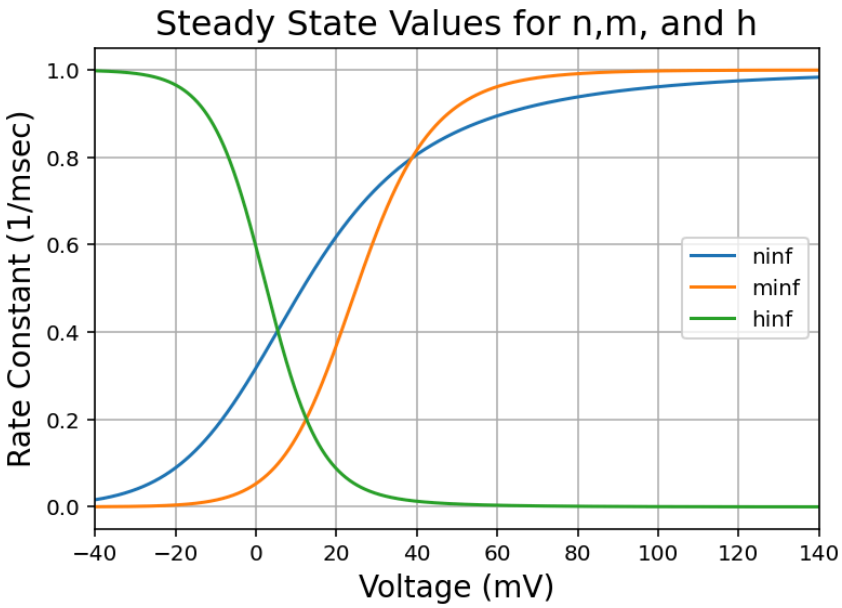

Figure 4 *Steady State Values for Potassium and Sodium Channel Gates*

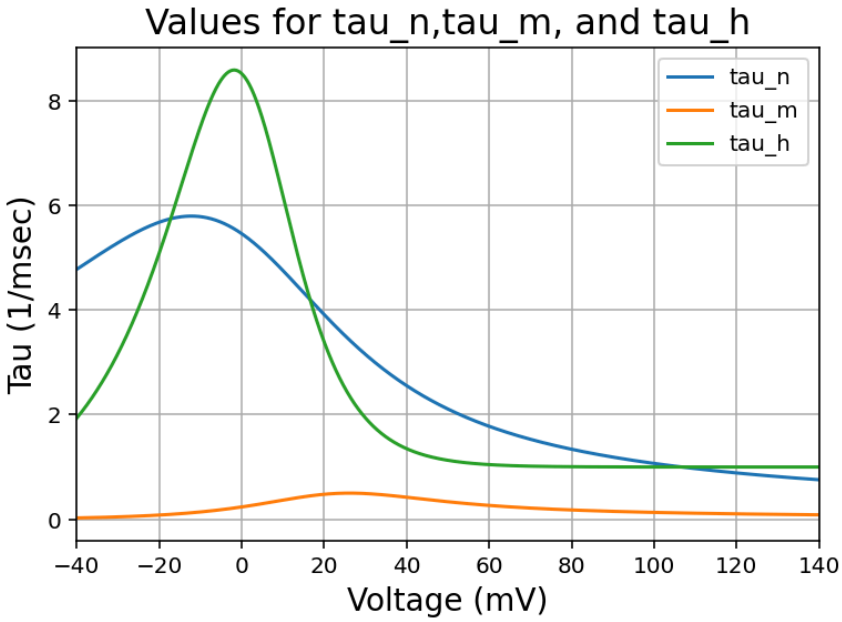

Figure 5 *Time Constants for Potassium and Sodium Channel Gates*

Why is the time constant for n slower at -40mV based on the rates of alpha-n and beta-n? At what negative potentials does it start to speed up and why?

The time constant for n is slower at -40mV because alpha_n and beta_n are low at -40mV according to Figure 1. Since tau for a given gate is one over the sum of the rates, then the lower rates would correspond to a higher tau meaning a slower response. At around -10mV tau_n begins to speed up because alpha_n begins to rapidly dominate the potassium gates. This means that around -10mV the n gates begin opening causing a rapid decrease in the time constant for n.

Problem 1c

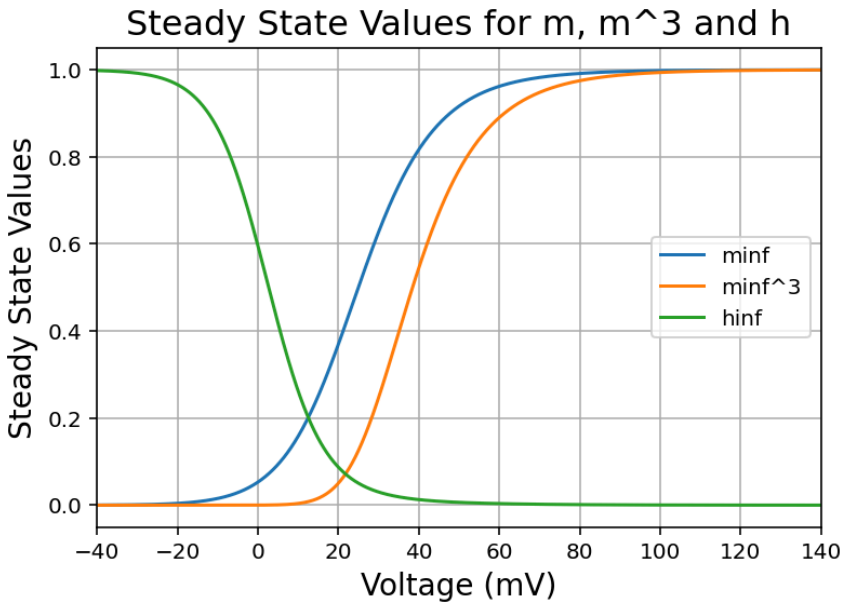

Figure 6 *Steady State Values of Sodium Activation (m) and Inactivation Gate (h)*

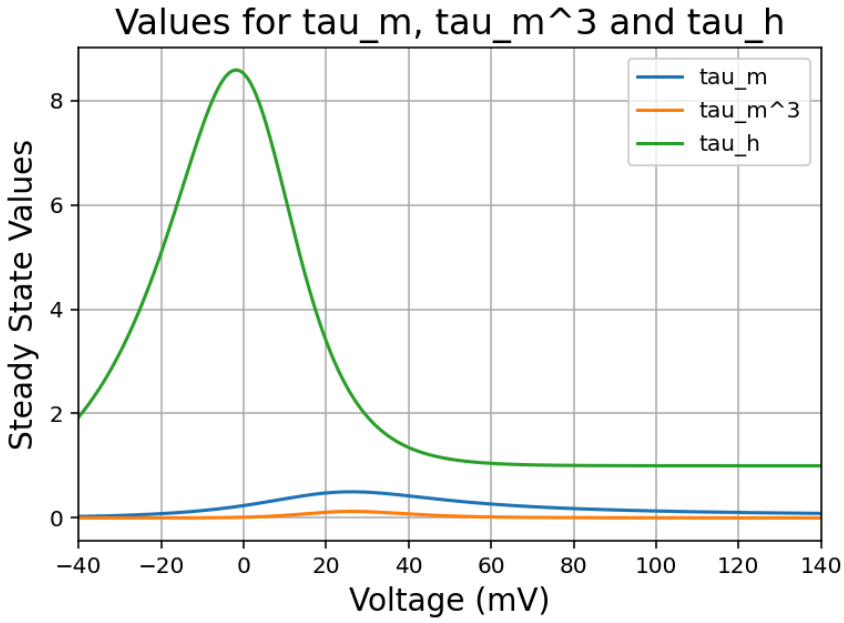

Figure 7 *Time Constants for Sodium Activation and Inactivation Gates*

At what range of voltages is the Sodium Current always being conducted?

Within the range of about 10mV to 30mV the sodium currents are always being conducted. This is because the steady state value of m cubed crosses the steady state value of h in that region as shown in Figure 6, and since that crossing occurs it means both gates are somewhat activated. Therefore current can flow through because m gates have been activated, but the sodium channels are not completely inactivated by the h gates yet.

Problem 1d

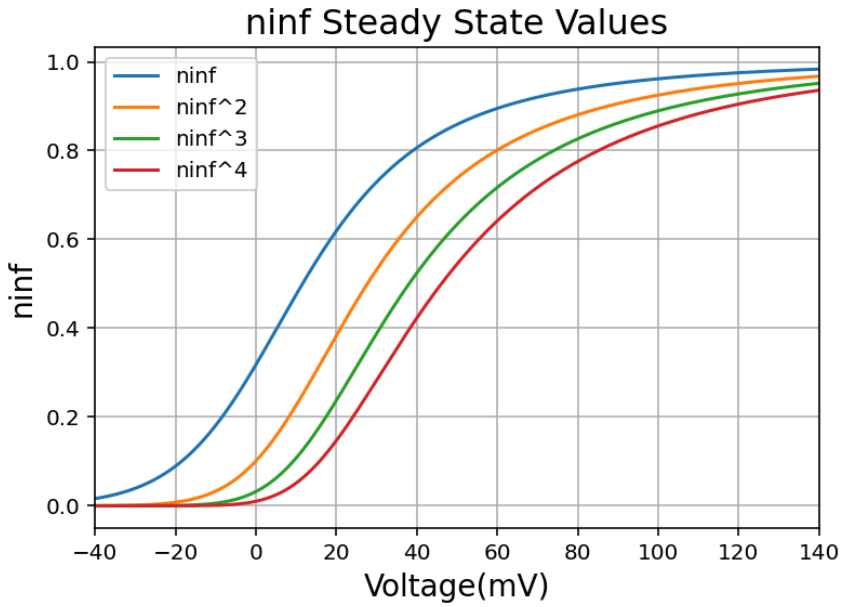

Figure 8 *Steady State Values for Potassium Channel Gate (n).*

What does the added exponentials do to the shapes of the curves?

The added exponentials shift the curves right as shown in Figure 8. Also along with the shift the shape becomes more sigmoidal which is consistent with the Hodgkin-Huxley model, truly showing that 4 n gates are needed to activate a potassium channel.

Problem 2a

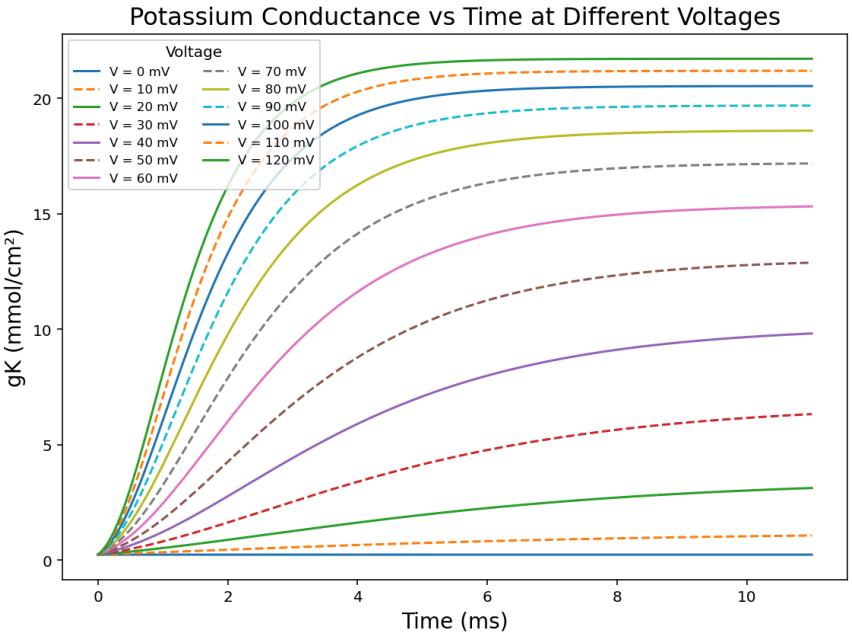

Figure 9 *Potassium Conductance for 10mV steps in voltage*

Problem 2b

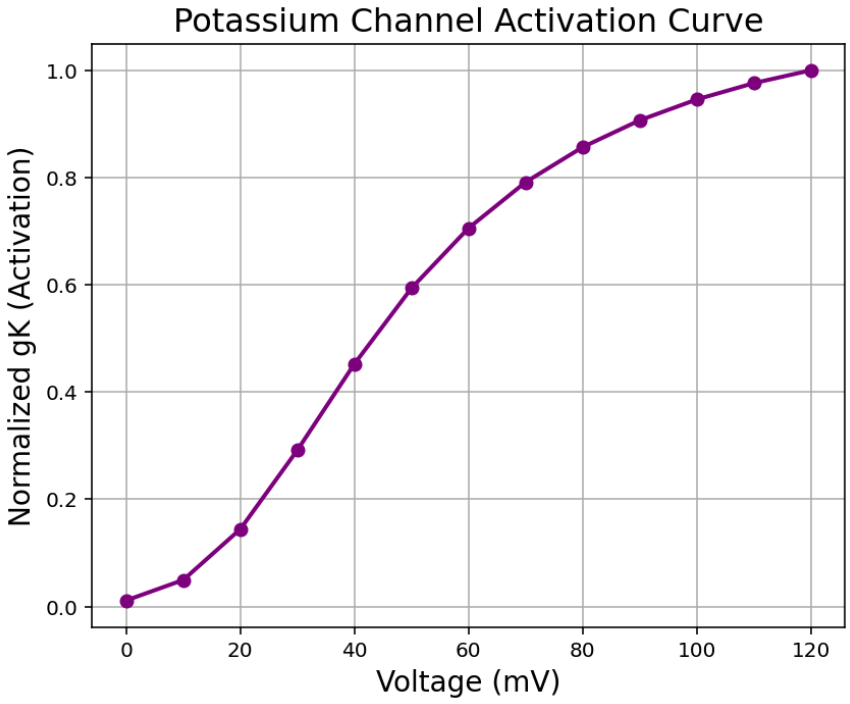

FIgure 10 *Activation of Potassium Channel using a Normalized Conductance*

Problem 2c

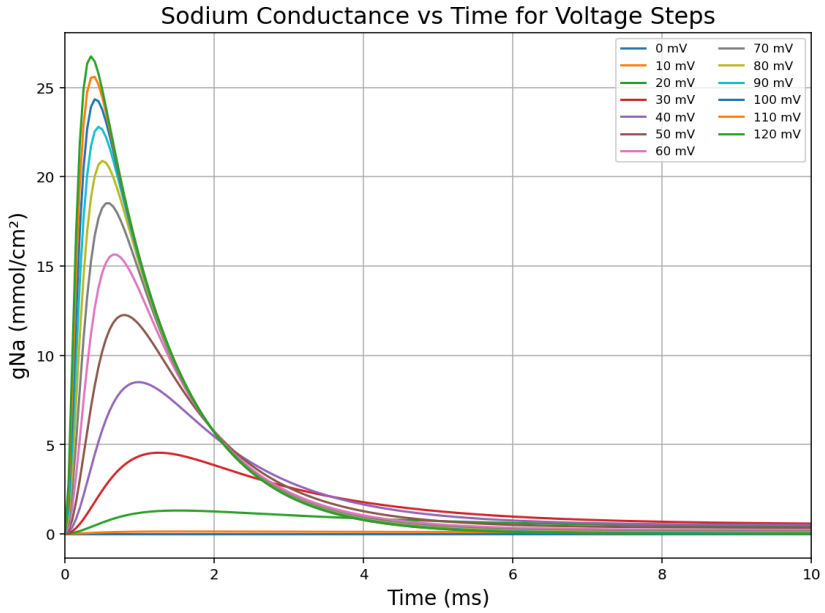

Figure 11 *Sodium Conductance Traces for 10mV steps in Voltage*

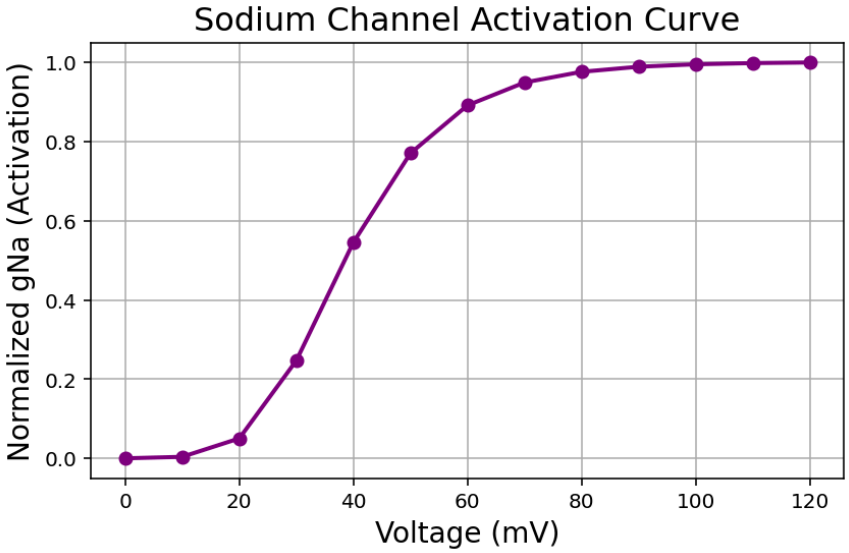

Figure 12 *Sodium Channel Activation Using Normalized Sodium Conductance*

Problem 2d

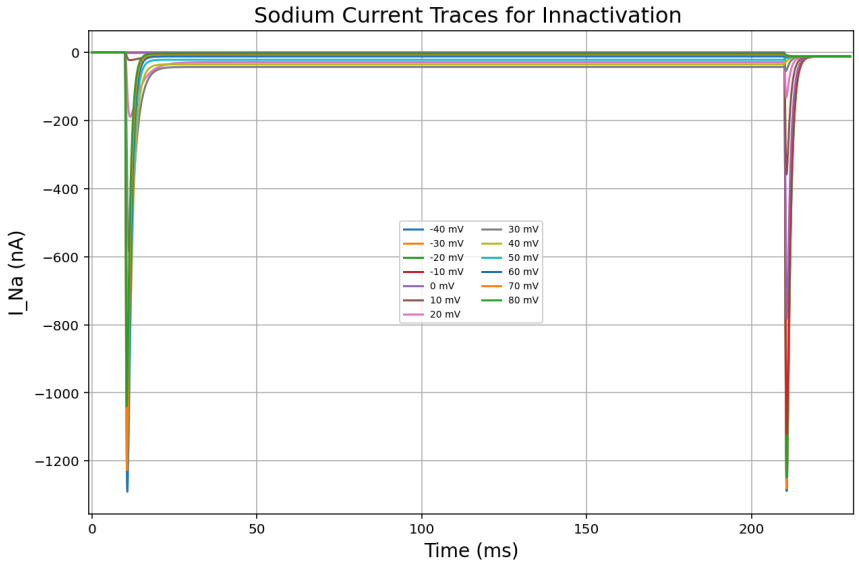

Figure 13 *Sodium Current for an inactivation protocol; showing the full 220ms*

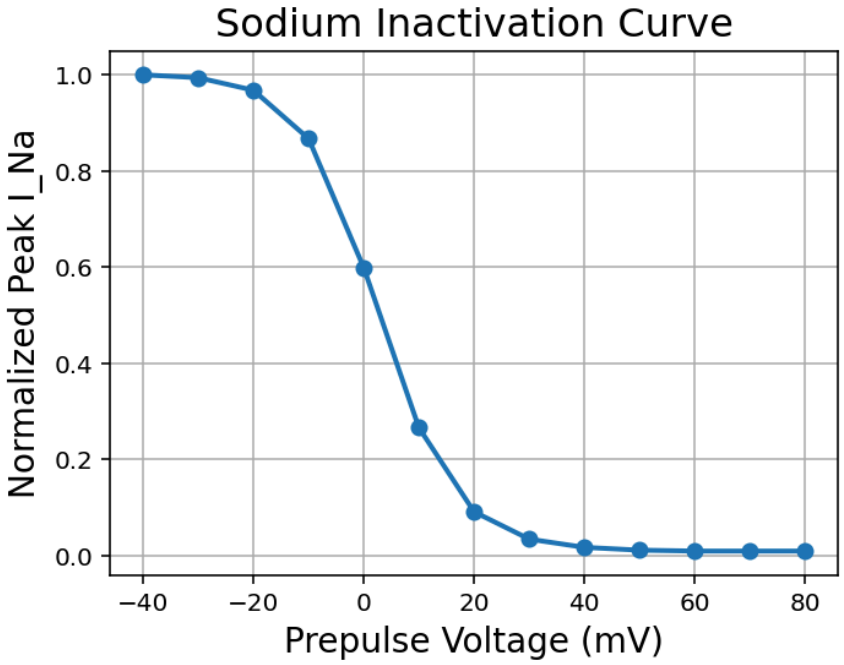

Figure 14 *Inactivation Curve for Sodium using a Normalized Peak Current*

Problem 2e

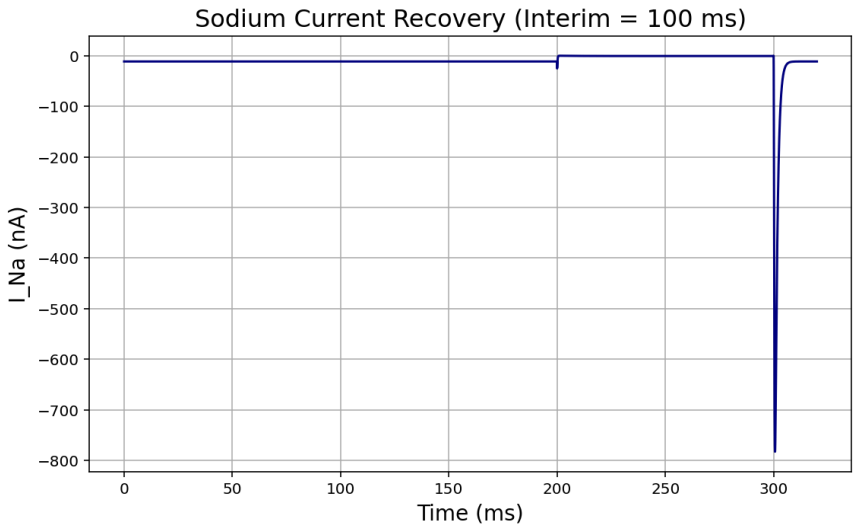

Figure 15 *Sodium Current for a 100ms Duration between the Prepulse and Testpulse using the recovery voltage protocol*

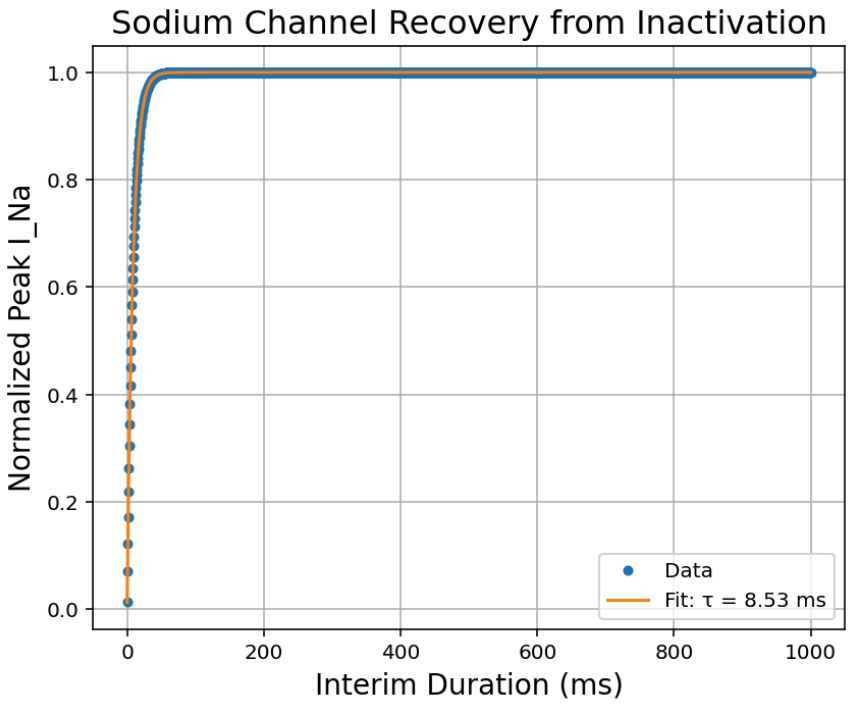

Figure 16 *Sodium recovery from inactivation using a normalized peak sodium current. The time constant was determined to be 8.53ms using an exponential fit.*

What is the time constant for channel recovery?

The time constant for channel recovery is 8.53 ms.

Problem 3a

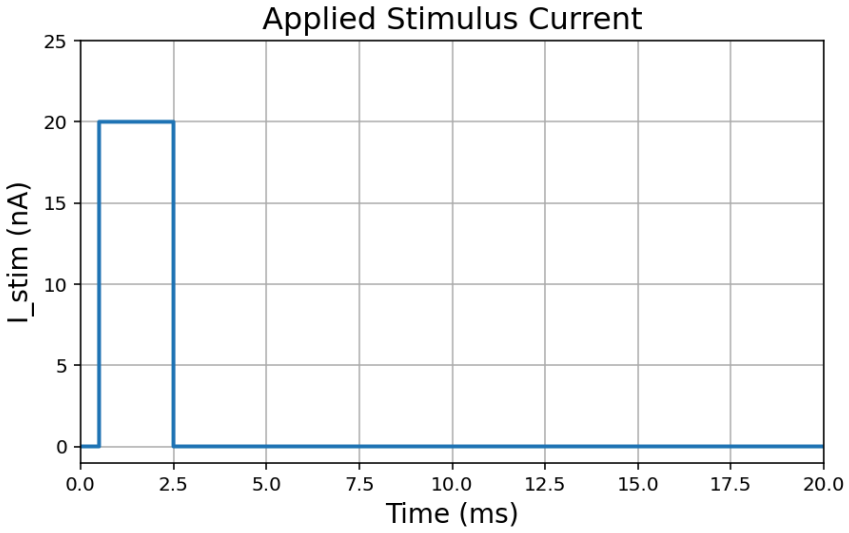

Figure 17 *Applied stimulus current to induce an action potential*

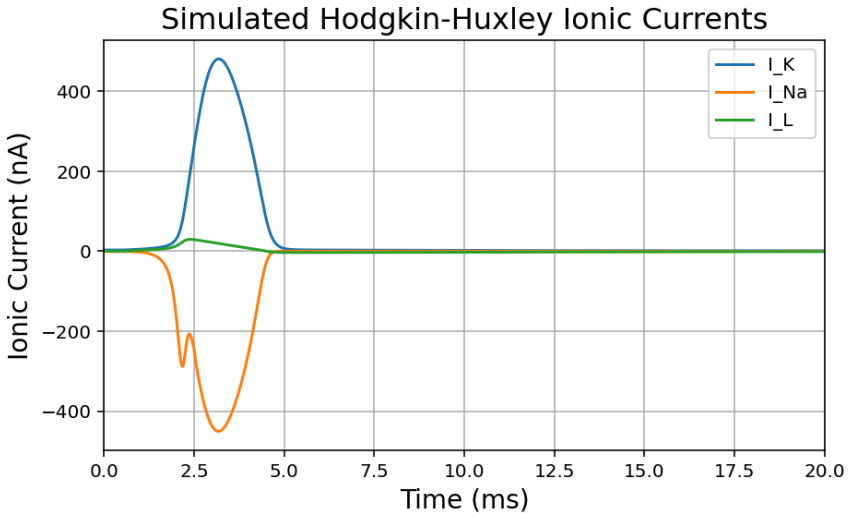

Figure 18 *Ionic current for a simulated action potential using the Hodgkin-Huxley model*

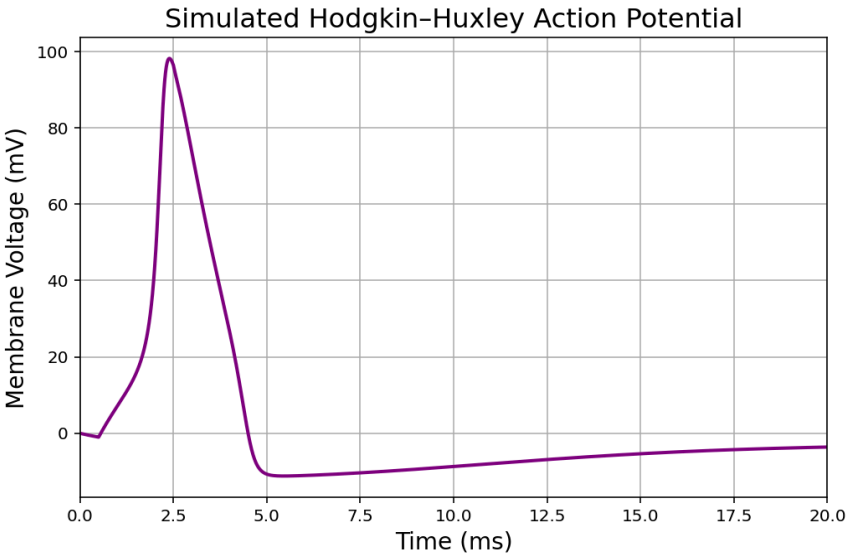

Figure 19 *Action potential induced by the stimulus in figure 17.*

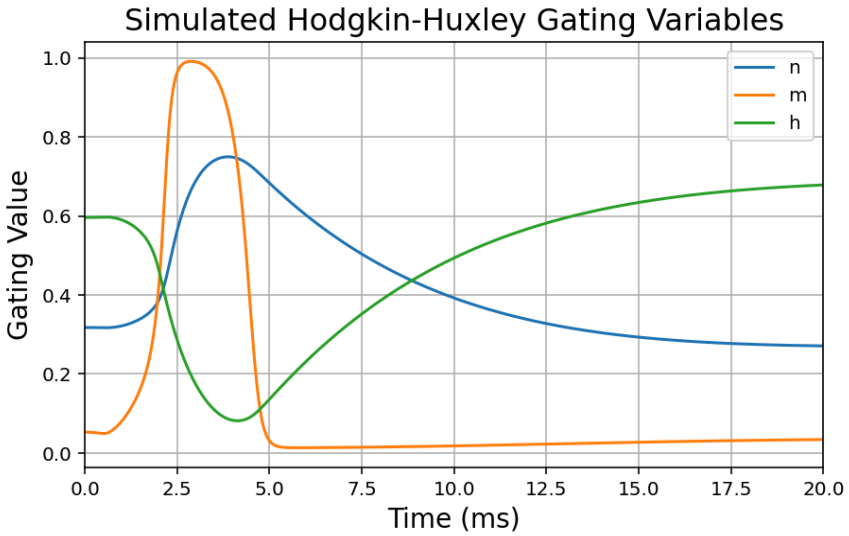

Figure 20 *Sodium and potassium gates governing the action potential in figure 19*

Problem 3b

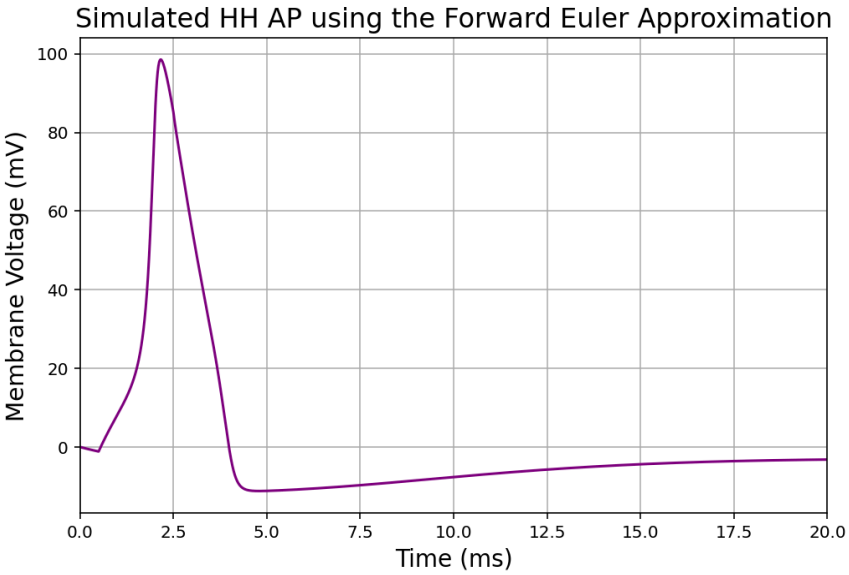

Figure 21 *Hodkin-Huckley action potential using the Euler approximation method*

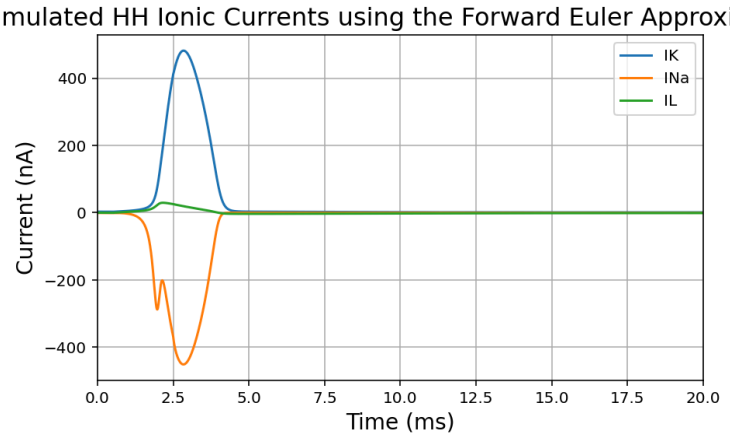

Figure 22 *Hodkin-Huxley ionic currents during an action potential while using the Euler approximation method*

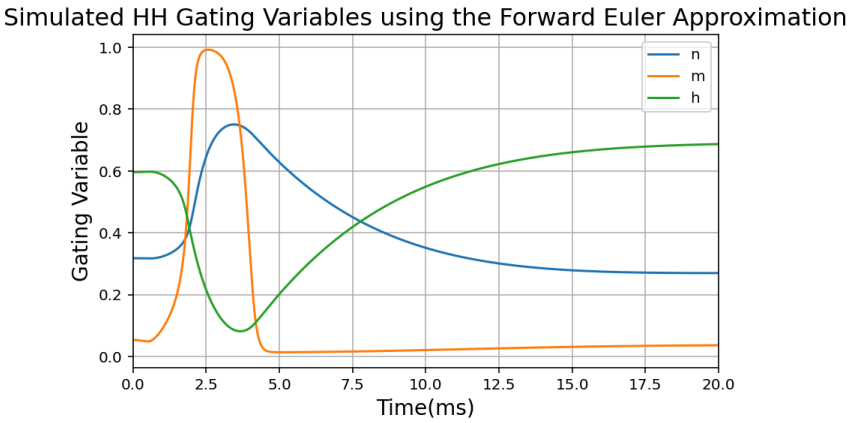

FIgure 23 *Hodkin-Huxley gating variables during an action potential while using the Euler approximation method*

What time step do you have to use to accurately simulate the action potential?

A time step of 0.0115ms was used to stimulate the action potential while using the Euler approximation method. This was accomplished by manually changing the step size value in order to obtain an action potential that closely represented the action potential whenever using the ODE solver in python.

Problem 3c

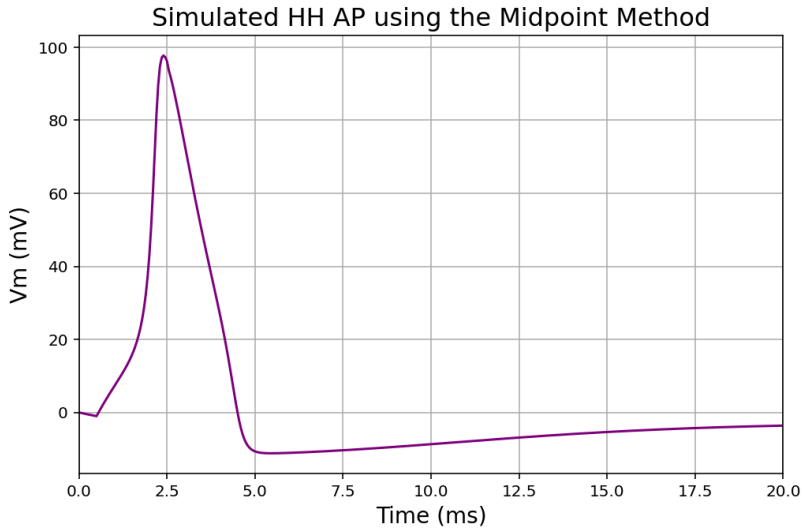

Figure 24 *Hodkin-Huckley action potential using the midpoint approximation method*

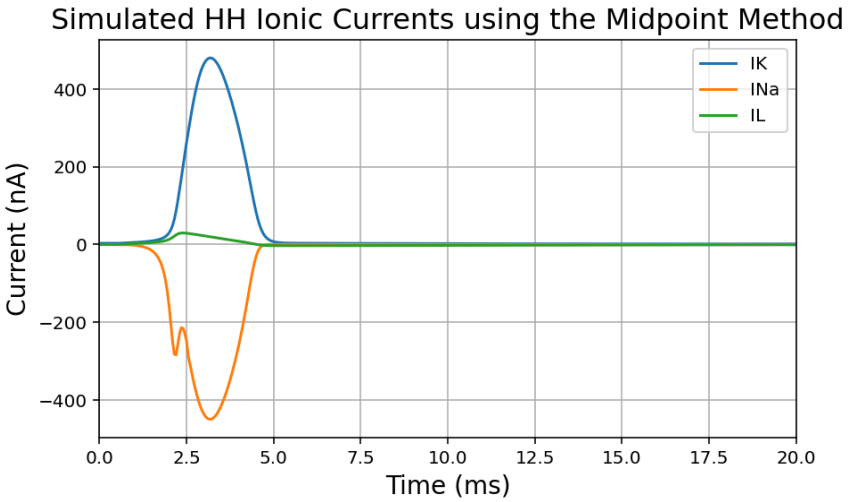

Figure 25 *Hodkin-Huxley ionic currents during an action potential while using the midpoint approximation method*

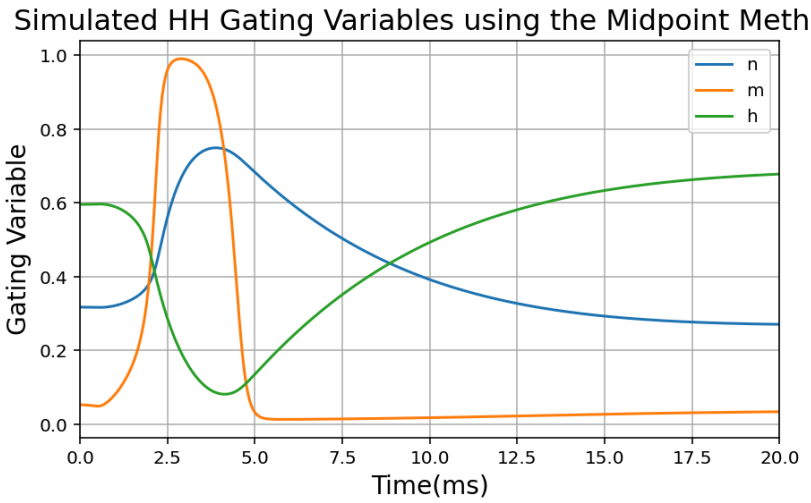

Figure 26 *Hodkin-Huxley gating variables during an action potential while using the midpoint approximation method*

How much bigger can you make the timestep, compared to Euler? Is it more efficient?

A time step of 0.05 ms was used to stimulate an action potential while using the midpoint method approximation. The midpoint method is definitely more accurate than the Euler approximation method because it utilizes calculating the slope at the midpoint of the time step instead of just at the beginning of the time step. The midpoint method is also more efficient than the Euler method because a bigger time step is used to replicate the action potential obtained when using the ODE solver.

Problem 4a

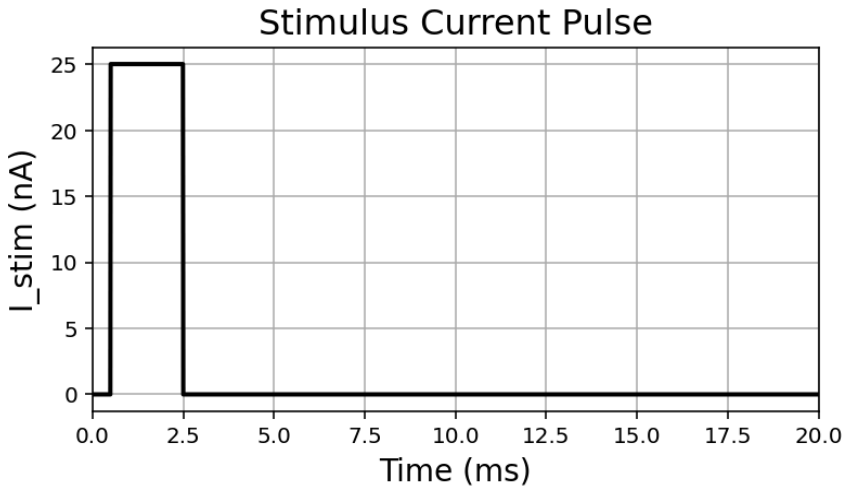

Figure 27 *Stimulus current needed to cause an action potential with 25% and 50% reduced GNa. As compared to figure 17 a higher pulse is required with reduced gNa. A pulse of 25nA was needed to simulate an action potential under the Sodium variant condition.*

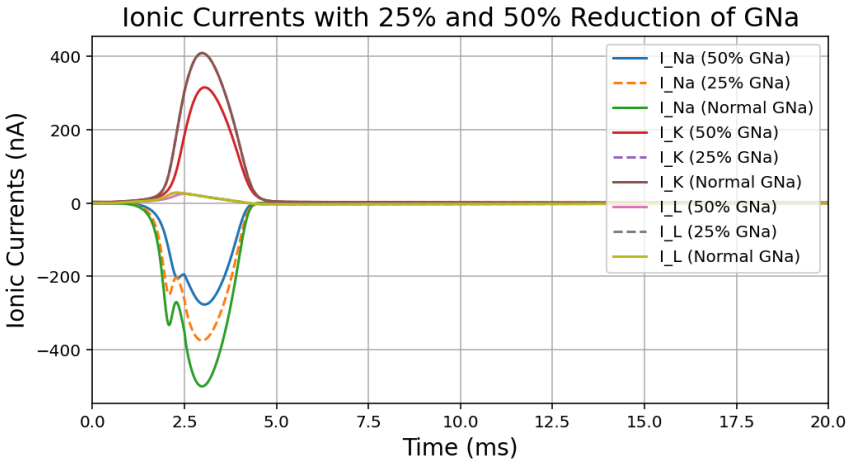

Figure 28 *Ionic currents governing an action potential with a Normal, 25% and 50% gNa*

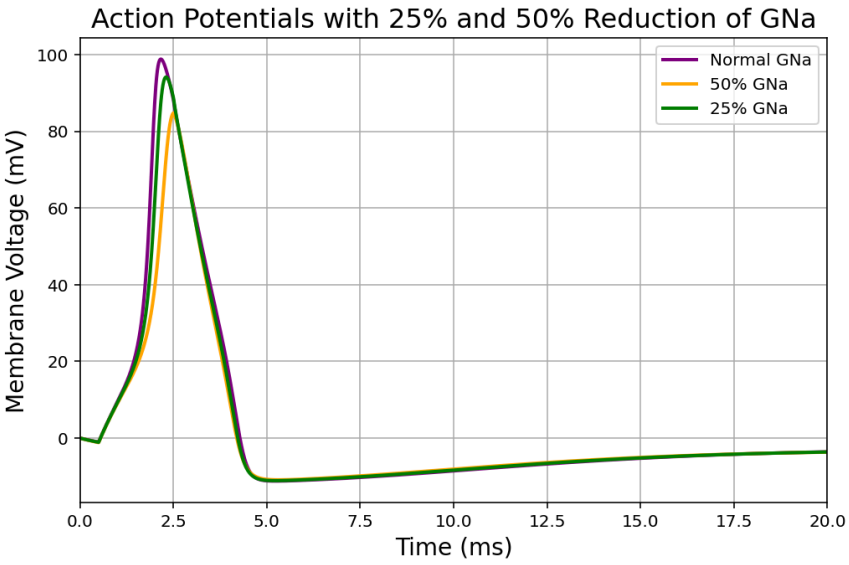

Figure 29 *Action potentials for Normal, 25% and 50% gNa*

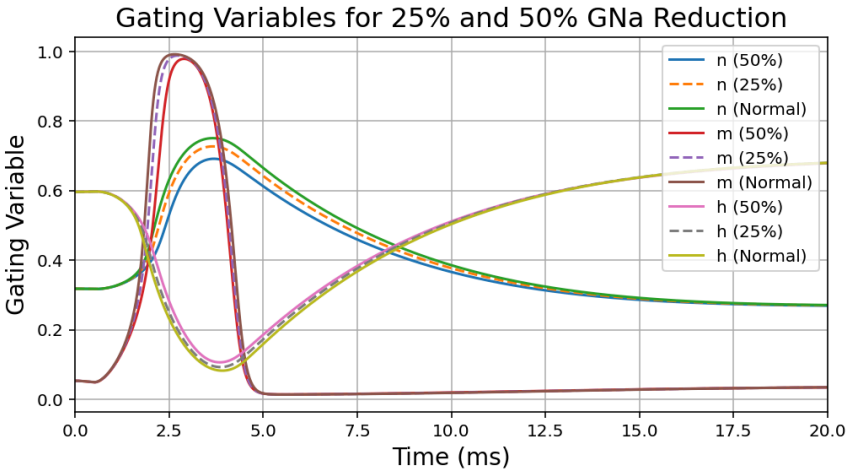

Figure 30 *Gating variables for an action potential that has Normal, 25% and 50% gNa*

How does the variant affect the amount of current needed to induce an action potential?

As shown in Figure 27 the stimulus current needed to cause an action potential with a sodium variant is higher than if the conductance of sodium was kept the same. By lowering the sodium conductance the inward Na current will be less, so there will be less depolarization, meaning that more stimulus is needed to overcome the threshold voltage to fire an action potential.

Problem 4b

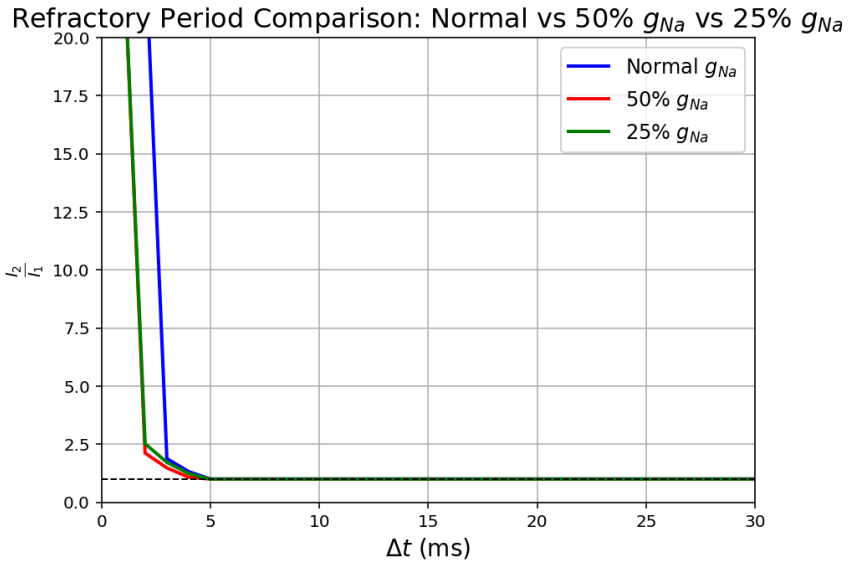

Figure 31*Refractory period comparison for Normal, 25% and 50% gNa*

Does it affect the refractory period? Why or why not?

As shown in Figure 30 the delta t for the variants is less than that of a non-variant condition. This shows that the sodium channels recover more quickly after the first pulse in the variant cases. Therefore the refractory period for the variants is a shorter time span between pulses due to the fact that sodium has the ability to recover faster. Also since the reduced gNa will cause the action potential to not reach as high of a potential then less repolarization is needed, and therefore the refractory period is shorter.

Problem 4c

Based on your simulation results, how does the variant affect neuronal excitability?

Based on the stimulation results, the variant affects the sodium channels themselves. Specifically when the sodium channel is recovering from inactivation which shortens the refractory period. Meaning the neuron can fire sooner which supports high frequency firing. At the same time, the variants require more injected current to stimulate an action potential, which slightly decreases excitability for individual spikes. Overall the neurons become less excitable for their initial excitation, but more excitable for a high frequency firing scenario.

Problem 4d

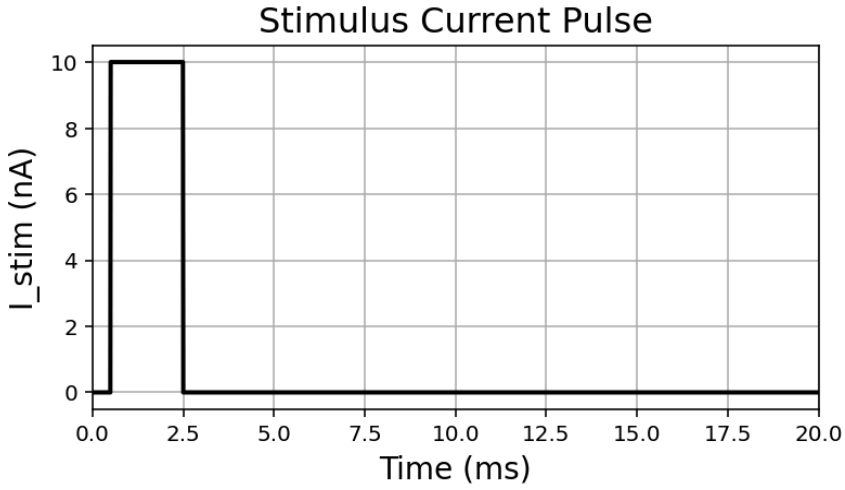

Figure 32 *Stimulus pulse needed to induce an action potential with 25% and 50% reduced gK. As compared to figure 17 this stimulus is less than the stimulus needed without the variant. A stimulus of 10nA was used to simulate an action potential under the Potassium variant conditions.*

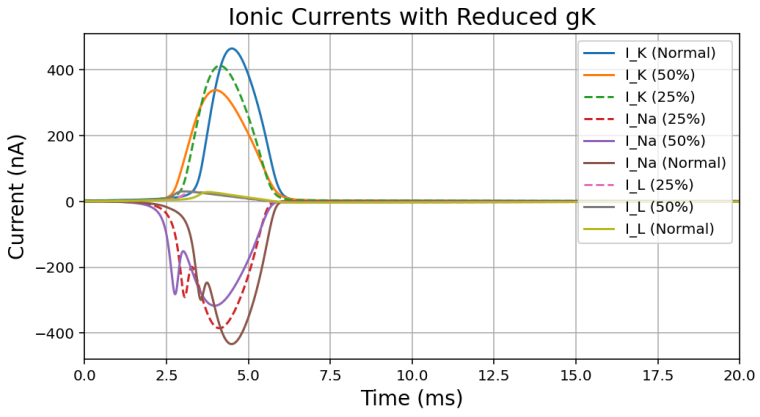

Figure 33 *Ionic currents for Normal, 25% and 50% gK*

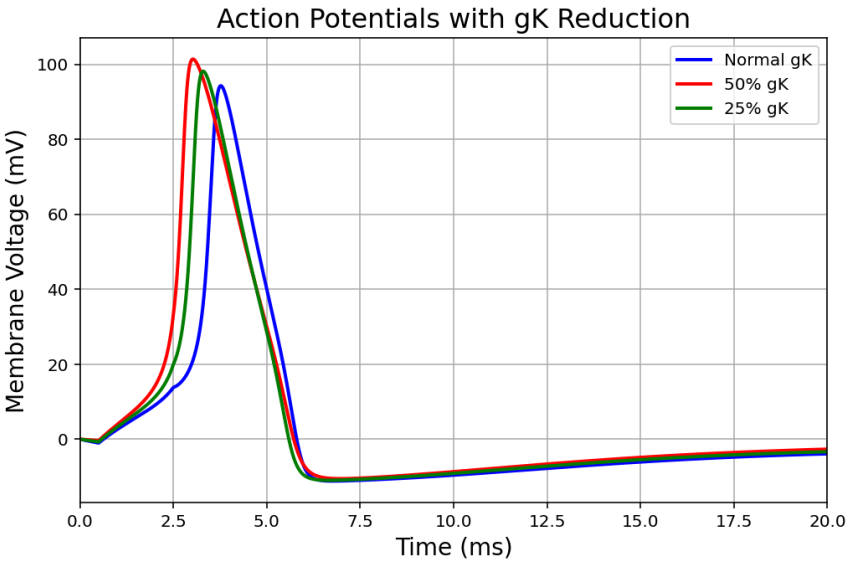

Figure 34 *Action Potential for Normal, 25% and 50% gK*

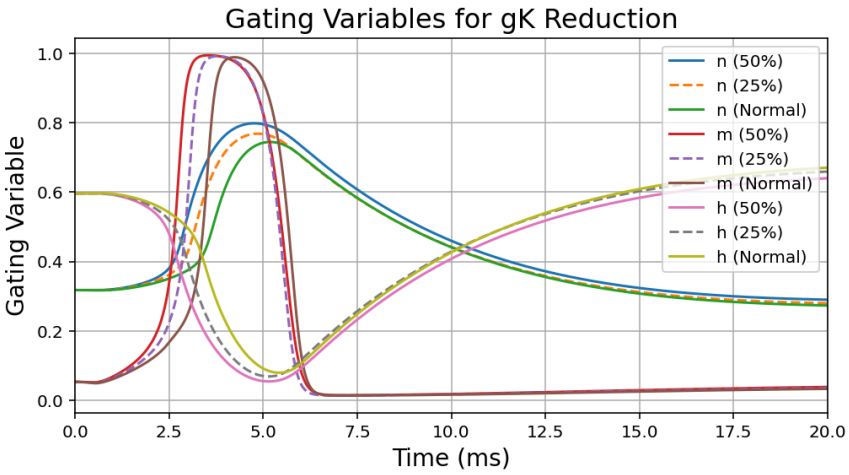

Figure 35 *Gating variables for Normal, 25% and 50% gK*

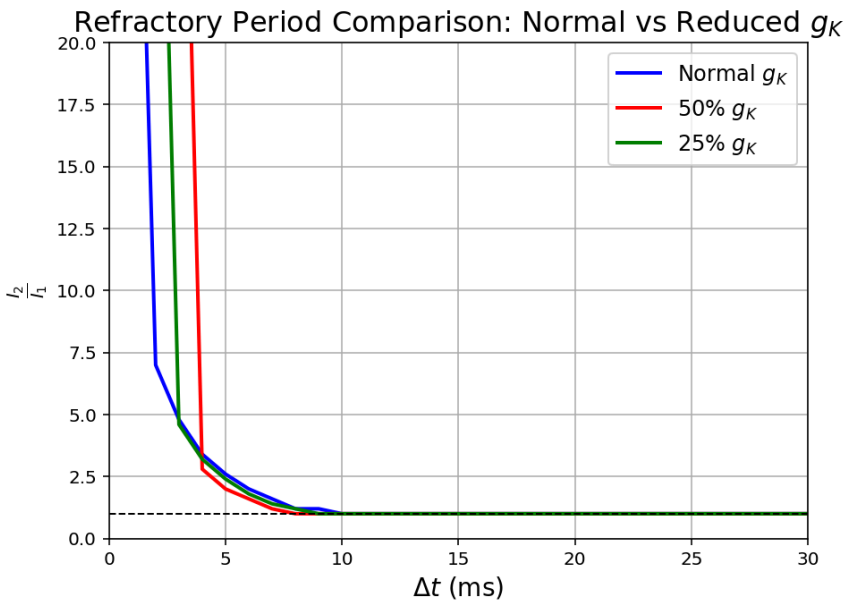

Figure 36 Refractory comparison *Normal, 25% and 50% gK*

How are the refractory period and amount of current needed to create an action potential by a variant that reduces Potassium conductance by 25 or 50%? Based on your simulations, do you predict that this type of Potassium channel variant will affect neuronal excitability differently than the Sodium channel variant that alters peak current?

Less stimulus is needed to induce an action potential with the potassium variant. Reducing potassium conductance by 25–50% slows repolarization, which lengthens the refractory period but reduces the current needed to trigger an action potential, since less outward K^+ opposes depolarization. In contrast, a sodium channel variant that reduces peak Na^+ current increases the threshold current but shortens the refractory period. Thus, potassium and sodium variants affect excitability differently: the potassium variant mainly changes recovery timing, while the sodium variant mainly changes threshold and spike initiation.

Unofficial Sources

I collaborated with Graham Uldrich and Sam Thompson while working on the Clab. ChatGPT was used for python code debugging.

Appendix A
Python Code for Problem 1

```
# Problem 1a
import numpy as np
import matplotlib.pyplot as plt

V = np.linspace(-40, 140, 2000)

import numpy as np

# FUnction to Calulate the Rates
def rates(Vm):
    # Compute alpha_n and beta_n
    if Vm == -10:  # usually the special case for n gating variable is at Vm = -10 mV
        a_n = 0.1
    else:
        a_n = (10 - Vm) / (100 * (np.exp((10 - Vm) / 10) - 1))
    b_n = 0.125 * np.exp(-Vm / 80)

    # Compute alpha_m and beta_m
    if Vm == -25:  # special case for m gating variable
        a_m = 1
    else:
        a_m = (25 - Vm) / (10 * (np.exp((25 - Vm) / 10) - 1))
    b_m = 4 * np.exp(-Vm / 18)

    # Compute alpha_h and beta_h
    a_h = 0.07 * np.exp(-Vm / 20)
    b_h = 1 / (np.exp((30 - Vm) / 10) + 1)

    return [a_n, b_n, a_m, b_m, a_h, b_h]


a_n = []
b_n = []
a_m = []
b_m = []
a_h = []
b_h = []
```

```
# For Loop to Creat Rates arrays
for v in V:
    soln = rates(v)
    a_n.append(soln[0])
    b_n.append(soln[1])
    a_m.append(soln[2])
    b_m.append(soln[3])
    a_h.append(soln[4])
    b_h.append(soln[5])

# Plotting the Rates as functions of Voltage
plt.plot(V, a_n, label = 'alpha_n')
plt.plot(V, b_n, label = 'beta_n')
plt.title('n Gate rate Constants', fontsize = 16)
plt.xlabel('Voltage (mV)', fontsize = 14)
plt.ylabel('Rate Constant (1/msec)', fontsize = 14)
plt.xlim(-40,140)
plt.grid(True)
plt.legend()
plt.show()
plt.plot(V, a_m, label = 'alpha_m')
plt.plot(V, b_m, label = 'beta_m')
plt.title('m Gate rate Constants',fontsize = 16)
plt.xlabel('Voltage (mV)',fontsize = 14)
plt.ylabel('Rate Constant (1/msec)',fontsize = 14)
plt.grid(True)
plt.xlim(-40,140)
plt.legend()
plt.show()
plt.plot(V, a_h, label = 'alpha_h')
plt.plot(V, b_h, label = 'beta_h')
plt.title('h Gate rate Constants',fontsize = 16)
plt.xlabel('Voltage (mV)',fontsize = 14)
plt.ylabel('Rate Constant (1/msec)',fontsize = 14)
plt.grid(True)
plt.xlim(-40,140)
plt.legend()
```

 # Problem 1b

```
import numpy as np
import matplotlib.pyplot as plt

V = np.linspace(-40, 140, 2000)

import numpy as np

# Function to Caluclate Rates
def rates(Vm):
    # Compute alpha_n and beta_n
    if Vm == -10:  # usually the special case for n gating variable is at Vm = -10 mV
        a_n = 0.1
    else:
        a_n = (10 - Vm) / (100 * (np.exp((10 - Vm) / 10) - 1))
    b_n = 0.125 * np.exp(-Vm / 80)

    # Compute alpha_m and beta_m
    if Vm == -25:  # special case for m gating variable
        a_m = 1
    else:
        a_m = (25 - Vm) / (10 * (np.exp((25 - Vm) / 10) - 1))
    b_m = 4 * np.exp(-Vm / 18)

    # Compute alpha_h and beta_h
    a_h = 0.07 * np.exp(-Vm / 20)
    b_h = 1 / (np.exp((30 - Vm) / 10) + 1)

    return [a_n, b_n, a_m, b_m, a_h, b_h]


ninf = []
minf = []
hinf = []
tau_n = []
tau_m = []
tau_h = []


# Function to Calulate Steady State Gating Values
def SS (a_n,b_n,a_m,b_m,a_h,b_h):
```

```
    minf = a_m/(b_m+a_m)
    hinf = a_h/(b_h+a_h)
    ninf = a_n/(b_n+a_n)
    return [ninf,minf,hinf]
# Function to Caluclate Taus
def tau (a_n,b_n,a_m,b_m,a_h,b_h):
    tau_n = 1/(a_n+b_n)
    tau_m = 1/(a_m+b_m)
    tau_h = 1/(a_h+b_h)
    return [tau_n,tau_m,tau_h]
# Loop to Append Arrays for Plotting
for v in V:
    soln = rates(v)
    a_n = soln[0]
    b_n = soln[1]
    a_m = soln[2]
    b_m = soln[3]
    a_h = soln[4]
    b_h = soln[5]

    soln1 = SS(a_n,b_n,a_m,b_m,a_h,b_h)
    soln2 = tau(a_n,b_n,a_m,b_m,a_h,b_h)
    ninf.append(soln1[0])
    minf.append(soln1[1])
    hinf.append(soln1[2])
    tau_n.append(soln2[0])
    tau_m.append(soln2[1])
    tau_h.append(soln2[2])
# Plot SS Values
plt.plot(V,ninf, label = 'ninf')
plt.plot(V,minf, label = 'minf')
plt.plot(V,hinf, label = 'hinf')
plt.title('Steady State Values for n,m, and h',fontsize = 16)
plt.ylabel('Rate Constant (1/msec)',fontsize = 14)
plt.xlabel('Voltage (mV)',fontsize = 14)
plt.grid(True)
plt.xlim(-40,140)
plt.legend()
plt.show()
# Plot Taus
```

```
plt.plot(V,tau_n, label = 'tau_n')
plt.plot(V,tau_m, label = 'tau_m')
plt.plot(V,tau_h, label = 'tau_h')
plt.title('Values for tau_n,tau_m, and tau_h',fontsize = 16)
plt.ylabel('Tau (1/msec)',fontsize = 14)
plt.xlabel('Voltage (mV)',fontsize = 14)
plt.grid(True)
plt.xlim(-40,140)
plt.legend()
plt.show()
# Problem 1c
import numpy as np
import matplotlib.pyplot as plt

V = np.linspace(-40, 140, 2000)

import numpy as np

# Function to Calculate Rates
def rates(Vm):
    # Compute alpha_n and beta_n
    if Vm == -10:  # usually the special case for n gating variable is at Vm = -10 mV
        a_n = 0.1
    else:
        a_n = (10 - Vm) / (100 * (np.exp((10 - Vm) / 10) - 1))
    b_n = 0.125 * np.exp(-Vm / 80)

    # Compute alpha_m and beta_m
    if Vm == -25:  # special case for m gating variable
        a_m = 1
    else:
        a_m = (25 - Vm) / (10 * (np.exp((25 - Vm) / 10) - 1))
    b_m = 4 * np.exp(-Vm / 18)

    # Compute alpha_h and beta_h
    a_h = 0.07 * np.exp(-Vm / 20)
    b_h = 1 / (np.exp((30 - Vm) / 10) + 1)

    return [a_n, b_n, a_m, b_m, a_h, b_h]
```

```
ninf = []
minf = []
hinf = []
tau_n = []
tau_m = []
tau_h = []


  # Function to Calulate Steady State Gating Values
def SS (a_n,b_n,a_m,b_m,a_h,b_h):
    minf = a_m/(b_m+a_m)
    hinf = a_h/(b_h+a_h)
    ninf = a_n/(b_n+a_n)
    return [ninf,minf,hinf]
# Function to Caluclate Taus
def tau (a_n,b_n,a_m,b_m,a_h,b_h):
    tau_n = 1/(a_n+b_n)
    tau_m = 1/(a_m+b_m)
    tau_h = 1/(a_h+b_h)
    return [tau_n,tau_m,tau_h]
# Appending Arrays for Plotting
for v in V:
    soln = rates(v)
    a_n = soln[0]
    b_n = soln[1]
    a_m = soln[2]
    b_m = soln[3]
    a_h = soln[4]
    b_h = soln[5]

    soln1 = SS(a_n,b_n,a_m,b_m,a_h,b_h)
    soln2 = tau(a_n,b_n,a_m,b_m,a_h,b_h)
    ninf.append(soln1[0])
    minf.append(soln1[1])
    hinf.append(soln1[2])
    tau_n.append(soln2[0])
    tau_m.append(soln2[1])
    tau_h.append(soln2[2])
# Create Arrays and Calulcate m^3 and tau^3
```

```
minf = np.array(minf)
tau_m = np.array(tau_m)
minf_3 = (minf**3.0)
taum_3 = (tau_m**3.0)

# Plot minf,minf^3, and h
plt.plot(V,minf, label = 'minf')
plt.plot(V,minf_3, label = 'minf^3')
plt.plot(V,hinf, label = 'hinf')
plt.title('Steady State Values for m, m^3 and h',fontsize = 16)
plt.xlabel('Voltage (mV)',fontsize = 14)
plt.ylabel('Steady State Values',fontsize = 14)
plt.grid(True)
plt.xlim(-40,140)
plt.legend()
plt.show()
# Plot taum,taum^3,and tauh
plt.plot(V,tau_m, label = 'tau_m')
plt.plot(V,taum_3, label = 'tau_m^3')
plt.plot(V,tau_h, label = 'tau_h')
plt.title('Values for tau_m, tau_m^3 and tau_h',fontsize = 16)
plt.xlabel('Voltage (mV)',fontsize = 14)
plt.ylabel('Steady State Values',fontsize = 14)
plt.grid(True)
plt.xlim(-40,140)
plt.legend()
plt.show()
```

 # Problem 1d

```
import numpy as np
import matplotlib.pyplot as plt

V = np.linspace(-40, 140, 2000)
import numpy as np

# Function to Calculate Rates
def rates(Vm):
    # Compute alpha_n and beta_n
    if Vm == -10:  # usually the special case for n gating variable is at Vm = -10 mV
        a_n = 0.1
```

```
    else:
        a_n = (10 - Vm) / (100 * (np.exp((10 - Vm) / 10) - 1))
    b_n = 0.125 * np.exp(-Vm / 80)

    # Compute alpha_m and beta_m
    if Vm == -25:  # special case for m gating variable
        a_m = 1
    else:
        a_m = (25 - Vm) / (10 * (np.exp((25 - Vm) / 10) - 1))
    b_m = 4 * np.exp(-Vm / 18)

    # Compute alpha_h and beta_h
    a_h = 0.07 * np.exp(-Vm / 20)
    b_h = 1 / (np.exp((30 - Vm) / 10) + 1)

    return [a_n, b_n, a_m, b_m, a_h, b_h]


ninf = []
minf = []
hinf = []
tau_n = []
tau_m = []
tau_h = []


 # Function to Calulate Steady State Gating Values
def SS (a_n,b_n,a_m,b_m,a_h,b_h):
    minf = a_m/(b_m+a_m)
    hinf = a_h/(b_h+a_h)
    ninf = a_n/(b_n+a_n)
    return [ninf,minf,hinf]
# Function to Caluclate Taus
def tau (a_n,b_n,a_m,b_m,a_h,b_h):
    tau_n = 1/(a_n+b_n)
    tau_m = 1/(a_m+b_m)
    tau_h = 1/(a_h+b_h)
    return [tau_n,tau_m,tau_h]

# Appending Arrays for Plotting
```

```
for v in V:
    soln = rates(v)
    a_n = soln[0]
    b_n = soln[1]
    a_m = soln[2]
    b_m = soln[3]
    a_h = soln[4]
    b_h = soln[5]

    soln1 = SS(a_n,b_n,a_m,b_m,a_h,b_h)
    soln2 = tau(a_n,b_n,a_m,b_m,a_h,b_h)
    ninf.append(soln1[0])
    minf.append(soln1[1])
    hinf.append(soln1[2])
    tau_n.append(soln2[0])
    tau_m.append(soln2[1])
    tau_h.append(soln2[2])

# Creating ninf array and calulating n^2,n^3, and n^4
ninf = np.array(ninf)
ninf_2 = ninf**2.0
ninf_3 = ninf**3.0
ninf_4 = ninf**4.0

# Plotting n,n^2,n^3, and n^4
plt.plot(V,ninf, label = 'ninf')
plt.plot(V,ninf_2, label = 'ninf^2')
plt.plot(V,ninf_3, label = 'ninf^3')
plt.plot(V,ninf_4, label = 'ninf^4')
plt.title('ninf Steady State Values',fontsize = 16)
plt.xlabel('Voltage(mV)',fontsize = 14)
plt.ylabel('ninf',fontsize = 14)
plt.grid(True)
plt.xlim(-40,140)
plt.legend()
plt.show()
```

Appendix B
Python Code for Problem 2

```
# Problem 2a
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
from scipy.optimize import curve_fit

V = np.linspace(0, 120, 13)
t = np.linspace(0,11,2000)
# Function to return dn/dt
def odefun (n,t,a_n,b_n):
    dndt = a_n*(1-n)-b_n*n
    return dndt

# Function to Calculate Rates
def rates(Vm):
    # Compute alpha_n and beta_n
    if Vm == 10:  # usually the special case for n gating variable is at Vm = -10 mV
        a_n = 0.1
    else:
        a_n = (10 - Vm) / (100 * (np.exp((10 - Vm) / 10) - 1))
    b_n = 0.125 * np.exp(-Vm / 80)

    # Compute alpha_m and beta_m
    if Vm == 25:  # special case for m gating variable
        a_m = 1
    else:
        a_m = (25 - Vm) / (10 * (np.exp((25 - Vm) / 10) - 1))
    b_m = 4 * np.exp(-Vm / 18)

    # Compute alpha_h and beta_h
    a_h = 0.07 * np.exp(-Vm / 20)
    b_h = 1 / (np.exp((30 - Vm) / 10) + 1)

    return [a_n, b_n, a_m, b_m, a_h, b_h]
# Function to Calculate SS for n,m, and h
def SS (a_n,b_n,a_m,b_m,a_h,b_h):
    minf = a_m/(b_m+a_m)
    hinf = a_h/(b_h+a_h)
```

```
    ninf = a_n/(b_n+a_n)
    return [ninf,minf,hinf]
# Calculate tau for n,m and h
def tau (a_n,b_n,a_m,b_m,a_h,b_h):
    tau_n = 1/(a_n+b_n)
    tau_m = 1/(a_m+b_m)
    tau_h = 1/(a_h+b_h)
    return [tau_n,tau_m,tau_h]
# Solve for n using odeint
def n (t, a_n, b_n):
    a_n_0,b_n_0 = rates(0)[0:2]
    n0 = a_n_0/(a_n_0+b_n_0)
    soln = odeint(odefun,n0,t, args = (a_n,b_n))
    return soln[:,0]
# Solve for gK
def gK (n):
    n = np.array(n)
    gk_bar = 24.01
    gK1 = gk_bar*(n**4.0)
    return gK1
# Plot gK for every voltage step; Every other line will be dotted
plt.figure(figsize=(8,6))
for i, v in enumerate(V):
    r = rates(v)
    n1 = n(t, r[0], r[1])
    gK1 = gK(n1)

    linestyle = '-' if i % 2 == 0 else '--'   # alternate by index
    plt.plot(t, gK1, linestyle=linestyle, label=f'V = {v:.0f} mV')

plt.xlabel('Time (ms)',fontsize = 14)
plt.ylabel('gK (mmol/cm²)',fontsize = 14)
plt.title('Potassium Conductance vs Time at Different Voltages',fontsize = 16)
plt.legend(title='Voltage', ncol=2, fontsize='small')
plt.tight_layout()
plt.show()
```

 # Problem 2b

```
gk_final = []
# Solve for n and gK
```

```
for v in V:
    r = rates(v)
    n1 = n(t, r[0], r[1])
    gK1 = gK(n1)
    gk_final.append(gK1[-1])

gK_final = np.array(gk_final)
# Normalize gK for Plotting
gK_norm = gk_final/np.max(gk_final)
# Plot Potassium Activation Curve
plt.figure(figsize=(6,5))
plt.plot(V, gK_norm, 'o-', color='purple', lw=2)
plt.xlabel('Voltage (mV)',fontsize = 14)
plt.ylabel('Normalized gK (Activation)',fontsize = 14)
plt.title('Potassium Channel Activation Curve',fontsize = 16)
plt.grid(True)
plt.tight_layout()
plt.show()
```

 # Problem 2c

```
tNa = np.linspace(0,100,2000)
# Solve for dm/dt
def odefun2 (m,tNa,a_m,b_m):
    dmdt = a_m*(1-m)-b_m*m
    return dmdt
# Solve for dh/dt
def odefun3(h,tNa,a_h,b_h):
    dhdt = a_h*(1-h)-b_h*h
    return dhdt
#solve for m
def m (tNa, a_m, b_m):
    a_m_0,b_m_0 = rates(0)[2:4]
    m0 = a_m_0/(a_m_0+b_m_0)
    soln = odeint(odefun2,m0,tNa, args = (a_m,b_m))
    return soln[:,0]
# Solve for h
def h (tNa,a_h,b_h):
    a_h_0,b_h_0 = rates(0)[4:6]
    h0 = a_h_0/(a_h_0+b_h_0)
    soln = odeint(odefun3,h0,tNa, args = (a_h,b_h))
```

```
    return soln[:,0]
# Solve for gNa
def gNa (m,h):
    m = np.array(m)
    h = np.array(h)
    gNa_bar = 70.7
    gNa1 = gNa_bar*(m**3)*h
    return gNa1
# Solve for gNa and plot for each voltage step
plt.figure(figsize=(8,6))
for v in V:
    r = rates(v)
    m1 = m(tNa, r[2], r[3])
    h1 = h(tNa, r[4], r[5])
    gNa1 = gNa(m1, h1)
    plt.plot(tNa, gNa1, label=f'{v:.0f} mV')

plt.xlabel('Time (ms)',fontsize = 14)
plt.ylabel('gNa (mmol/cm²)',fontsize = 14)
plt.title('Sodium Conductance vs Time for Voltage Steps',fontsize = 16)
plt.legend(ncol=2, fontsize='small')
plt.xlim(0, 10)
plt.grid(True)
plt.tight_layout()
plt.show()

# Na Activation Curve
gNa_final = []
for v in V:
    r = rates(v)
    m1 = m(tNa, r[2], r[3])
    gNa1 = m1**3
    gNa_final.append(gNa1[-1])



gNa_final = np.array(gNa_final)
# Normalize gNa for plotting
gNa_norm = gNa_final/np.max(gNa_final)

# Plot Na activation curve
```

```
plt.figure()
plt.plot(V, gNa_norm, 'o-', color='purple', lw=2)
plt.xlabel('Voltage (mV)',fontsize = 14)
plt.ylabel('Normalized gNa (Activation)',fontsize = 14)
plt.title('Sodium Channel Activation Curve',fontsize = 16)
plt.grid(True)
plt.tight_layout()
plt.show()
```

 # Problem 2d

```
V_inac = np.linspace(-40, 80, 13)  # prepulse voltages

# Protocol voltages
V_hold = -80.0    # holding potential (mV) before prepulse
V_test = 60.0     # test pulse (mV)

# Durations and time vectors (ms)
t_hold = np.linspace(0, 10, 200)    # 10 ms hold at V_hold to produce a resting onset
t_pulse = np.linspace(0, 200, 2000) # 200 ms prepulse
t_test  = np.linspace(0, 20, 400)   # 20 ms test pulse

E_Na = 110.0
gNa_bar = 70.7

INa_Peak = []

plt.figure(figsize=(9,6))

for v in V_inac:
    # --- 1) Start from hold at V_hold to ensure channels are at resting state and produce an onset
at prepulse ---
    a_m_hold, b_m_hold, a_h_hold, b_h_hold = rates(V_hold)[2:6]
    m0_hold = a_m_hold / (a_m_hold + b_m_hold)
    h0_hold = a_h_hold / (a_h_hold + b_h_hold)

    # integrate over hold to get initial condition at the moment of prepulse
    m_hold = odeint(odefun2, m0_hold, t_hold, args=(a_m_hold, b_m_hold))[:, 0]
    h_hold = odeint(odefun3, h0_hold, t_hold, args=(a_h_hold, b_h_hold))[:, 0]
```

```
    INa_hold = gNa_bar * (m_hold**3) * h_hold * (V_hold - E_Na)

    # --- 2) Prepulse to voltage v (this will produce the first transient peak at prepulse onset) ---
    a_m_pre, b_m_pre, a_h_pre, b_h_pre = rates(v)[2:6]
    m0_prep = m_hold[-1]
    h0_prep = h_hold[-1]
    m_prep = odeint(odefun2, m0_prep, t_pulse, args=(a_m_pre, b_m_pre))[:, 0]
    h_prep = odeint(odefun3, h0_prep, t_pulse, args=(a_h_pre, b_h_pre))[:, 0]
    INa_prep = gNa_bar * (m_prep**3) * h_prep * (v - E_Na)

    # --- 3) Step to test pulse (V_test) starting from end of prepulse (this gives the second
transient) ---
    a_m_test, b_m_test, a_h_test, b_h_test = rates(V_test)[2:6]
    m0_test = m_prep[-1]
    h0_test = h_prep[-1]
    m_test = odeint(odefun2, m0_test, t_test, args=(a_m_test, b_m_test))[:, 0]
    h_test = odeint(odefun3, h0_test, t_test, args=(a_h_test, b_h_test))[:, 0]
    INa_test = gNa_bar * (m_test**3) * h_test * (V_test - E_Na)

    #  combine time and currents  for plotting
    t_combined = np.concatenate([
        t_hold,
        t_hold[-1] + t_pulse,
        t_hold[-1] + t_pulse[-1] + t_test
    ])
    INa_combined = np.concatenate([INa_hold, INa_prep, INa_test])

    # record the test-peak magnitude for the inactivation curve
    INa_Peak.append(np.max(np.abs(INa_test)))

    # plot the full trace with label
    plt.plot(t_combined, INa_combined, label=f'{v:.0f} mV')


# plot layout
plt.xlim(-1, t_combined[-1] + 1)
plt.xlabel('Time (ms)', fontsize=14)
plt.ylabel('I_Na (nA)', fontsize=14)
plt.title('Sodium Current Traces for Innactivation ', fontsize=16)
plt.legend(ncol=2, fontsize='x-small', loc='center')
```

```
plt.grid(True)
plt.tight_layout()
plt.show()

# Inactivation curve
INa_Peak = np.array(INa_Peak)
INa_norm = INa_Peak / np.max(INa_Peak)

plt.figure(figsize=(5,4))
plt.plot(V_inac, INa_norm, 'o-', linewidth=2)
plt.xlabel("Prepulse Voltage (mV)", fontsize=14)
plt.ylabel("Normalized Peak I_Na (test)", fontsize=14)
plt.title("Sodium Inactivation Curve", fontsize=16)
plt.grid(True)
plt.tight_layout()
plt.show()
```

 # Problem 2e

```
V1 = 60
t1 = 200
V2 = 0
t2_values = np.linspace(0,1000,2000)
V3 = 60
t3 = 20
# Function to solve for m and h under the pulse protocol
def run_pulse(Vm, tspan, m0, h0):
    a_m, b_m, a_h, b_h = rates(Vm)[2:6]
    m_sol = odeint(odefun2, m0, tspan, args=(a_m, b_m))[:, 0]
    h_sol = odeint(odefun3, h0, tspan, args=(a_h, b_h))[:, 0]
    return m_sol, h_sol

INa_peaks = []

# Loop over durations and solve for needed values of n,m and h
for t2 in t2_values:
    # First pulse (to inactivate)
    tspan1 = np.linspace(0, t1, 1000)
```

```
    a_m1, b_m1, a_h1, b_h1 = rates(V1)[2:6]
    m_inf1 = a_m1 / (a_m1 + b_m1)
    h_inf1 = a_h1 / (a_h1 + b_h1)
    m0 = m_inf1  # assume steady state
    h0 = h_inf1

    m1, h1 = run_pulse(V1, tspan1, m0, h0)

    # Interim at 0 mV
    tspan2 = np.linspace(0, t2, max(2, int(t2 * 10 + 2)))  # variable resolution
    m2, h2 = run_pulse(V2, tspan2, m1[-1], h1[-1])

    # Second pulse at 60 mV
    tspan3 = np.linspace(0, t3, 1000)
    m3, h3 = run_pulse(V3, tspan3, m2[-1], h2[-1])

    # Sodium current during second pulse
    INa = gNa_bar * (m3**3) * h3 * (V3 - E_Na)
    INa_peaks.append(np.max(np.abs(INa)))

# Normalize recovery
INa_peaks = np.array(INa_peaks)
INa_norm = INa_peaks / np.max(INa_peaks)

# Exponential fit for recovery
def exp_recovery(t, tau, A):
    return 1 - A * np.exp(-t / tau)

popt, _ = curve_fit(exp_recovery, t2_values, INa_norm, p0=[10, 1])
tau_fit = popt[0]

# Plot single trace for t2 = 100 ms
t2_example = 100

#First pulse (V1 = 60 mV, 200 ms)
tspan1 = np.linspace(0, t1, 1000)
a_m1, b_m1, a_h1, b_h1 = rates(V1)[2:6]
m_inf1 = a_m1 / (a_m1 + b_m1)
h_inf1 = a_h1 / (a_h1 + b_h1)
m1, h1 = run_pulse(V1, tspan1, m_inf1, h_inf1)
```

```
INa1 = gNa_bar * (m1**3) * h1 * (V1 - E_Na)

# Interim (V2 = 0 mV, variable duration
tspan2 = np.linspace(0, t2_example, max(2, int(t2_example * 10 + 2)))
m2, h2 = run_pulse(V2, tspan2, m1[-1], h1[-1])
INa2 = gNa_bar * (m2**3) * h2 * (V2 - E_Na)   # basically ~0 nA

#Second pulse (V3 = 60 mV, 20 ms)
tspan3 = np.linspace(0, t3, 1000)
m3, h3 = run_pulse(V3, tspan3, m2[-1], h2[-1])
INa3 = gNa_bar * (m3**3) * h3 * (V3 - E_Na)

# Concatenate times and currents
t_full = np.concatenate([
    tspan1,
    t1 + tspan2,
    t1 + t2_example + tspan3
])
INa_full = np.concatenate([
    INa1,
    INa2,
    INa3
])


#Plot sodium current Recovery
plt.figure(figsize=(8,5))
plt.plot(t_full, INa_full, color='navy')
plt.xlabel('Time (ms)', fontsize=14)
plt.ylabel('I_Na (nA)', fontsize=14)
plt.title(f'Sodium Current Recovery (Interim = {t2_example} ms)', fontsize=16)
plt.grid(True)
plt.tight_layout()
plt.show()
# Plot recovery curve
plt.figure(figsize=(6,5))
plt.plot(t2_values, INa_norm, 'o', label='Data', markersize = 4)
plt.plot(t2_values, exp_recovery(t2_values, *popt), '-', label=f'Fit: τ = {tau_fit:.2f} ms')
plt.xlabel('Interim Duration (ms)',fontsize = 14)
plt.ylabel('Normalized Peak I_Na',fontsize = 14)
```

```
plt.title('Sodium Channel Recovery from Inactivation',fontsize = 16)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Appendix C
Python Code for Problem 3

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

# Constants
# Problem 3a
gNa_bar = 70.7
gK_bar  = 24.31
ENa = 110
EK = -12
Cm = 1
gL_bar = 0.3
EL = 0
t = np.linspace(0, 20, 2000)
# Function to solve for rates
def rates(Vm):
    if abs(Vm + 10) < 1e-6:
        a_n = 0.1
    else:
        a_n = (10 - Vm) / (100 * (np.exp((10 - Vm) / 10) - 1))
    b_n = 0.125 * np.exp(-Vm / 80)

    if abs(Vm + 25) < 1e-6:
        a_m = 1
    else:
        a_m = (25 - Vm) / (10 * (np.exp((25 - Vm) / 10) - 1))
    b_m = 4 * np.exp(-Vm / 18)

    a_h = 0.07 * np.exp(-Vm / 20)
    b_h = 1 / (np.exp((30 - Vm) / 10) + 1)

    return a_n, b_n, a_m, b_m, a_h, b_h
# Function to Output the stimulus current
def IStim(t):
    return 20 if 0.5 <= t <= 2.5 else 0
# Function to Solve the HH Equations
def derivatives(y, t, I_func):
    V, n, m, h = y
```

```
    a_n, b_n, a_m, b_m, a_h, b_h = rates(V)

    dndt = a_n*(1-n) - b_n*n
    dmdt = a_m*(1-m) - b_m*m
    dhdt = a_h*(1-h) - b_h*h

    IK  = gK_bar * n**4 * (V - EK)
    INa = gNa_bar * m**3 * h * (V - ENa)
    IL  = gL_bar * (V - EL)

    dVdt = (I_func(t) - (IK+INa+IL)) / Cm
    return [dVdt, dndt, dmdt, dhdt]

# Initial gating variables
a_n, b_n, a_m, b_m, a_h, b_h = rates(0)
# Compute initial gating variables at V = 0
a_n, b_n, a_m, b_m, a_h, b_h = rates(0)
n0 = a_n / (a_n + b_n)
m0 = a_m / (a_m + b_m)
h0 = a_h / (a_h + b_h)
y0 = [0, a_n/(a_n+b_n), a_m/(a_m+b_m), a_h/(a_h+b_h)]

sol = odeint(derivatives, y0, t, args=(IStim,))
V, n, m, h = sol.T

# Compute currents
IK  = gK_bar * n**4 * (V - EK)
INa = gNa_bar * m**3 * h * (V - ENa)
IL  = gL_bar * (V - EL)

# Plot Stimulus Current
I_vals = np.array([IStim(T) for T in t])  # Evaluate IStim over time vector

plt.figure(figsize=(7,4))
plt.step(t, I_vals, where='post', linewidth=2)
plt.title("Applied Stimulus Current",fontsize = 16)
plt.xlabel("Time (ms)",fontsize = 14)
plt.ylabel("I_stim (nA)",fontsize = 14)
plt.ylim(-1, max(I_vals) + 5)
plt.xlim(0,20)
```

```
plt.grid(True)
plt.show()

# Plot Ionic Currents
plt.figure(figsize=(7,4))
plt.plot(t, IK, label="I_K")
plt.plot(t, INa, label="I_Na")
plt.plot(t, IL, label="I_L")
plt.title("Simulated Hodgkin-Huxley Ionic Currents",fontsize = 16)
plt.xlabel("Time (ms)",fontsize = 14)
plt.ylabel("Ionic Current (nA)",fontsize = 14)
plt.xlim(0,20)
plt.legend()
plt.grid(True)
plt.show()
# Plot Action Potential
plt.figure(figsize=(8,5))
plt.plot(t, V, lw=2, color='purple')
plt.title("Simulated Hodgkin–Huxley Action Potential",fontsize = 16)
plt.xlabel("Time (ms)",fontsize = 14)
plt.ylabel("Membrane Voltage (mV)",fontsize = 14)
plt.grid(True)
plt.xlim(0,20)
plt.show()
# Plot Gating Variables
plt.figure(figsize=(7,4))
plt.plot(t, n, label="n")
plt.plot(t, m, label="m")
plt.plot(t, h, label="h")
plt.title("Simulated Hodgkin-Huxley Gating Variables",fontsize = 16)
plt.xlabel("Time (ms)",fontsize = 14)
plt.ylabel("Gating Value",fontsize = 14)
plt.xlim(0,20)
plt.legend()
plt.grid(True)
plt.show()
```

 # Problem 3b : Forward Euler

```
dt = 0.0115
y = np.array([0, n0, m0, h0], dtype=float)
```

```
V_FE = np.zeros_like(t)
n_FE = np.zeros_like(t)
m_FE = np.zeros_like(t)
h_FE = np.zeros_like(t)

IK_FE = np.zeros_like(t)
INa_FE = np.zeros_like(t)
IL_FE = np.zeros_like(t)

for i in range(len(t)):
    V_FE[i], n_FE[i], m_FE[i], h_FE[i] = y

    # Currents
    IK_FE[i]  = gK_bar * n_FE[i]**4 * (V_FE[i] - EK)
    INa_FE[i] = gNa_bar * m_FE[i]**3 * h_FE[i] * (V_FE[i] - ENa)
    IL_FE[i]  = gL_bar * (V_FE[i] - EL)

    # Forward Euler update
    y = y + dt * np.array(derivatives(y, t[i], IStim))


# Plot AP
plt.figure(figsize=(8,5))
plt.plot(t, V_FE, color='purple')
plt.title("Simulated HH AP using the Forward Euler Approximation",fontsize = 16)
plt.xlabel("Time (ms)",fontsize = 14)
plt.ylabel("Membrane Voltage (mV)",fontsize = 14)
plt.xlim(0,20)
plt.grid(True)
plt.show()

# Plot Ionic Currents
plt.figure(figsize=(7,4))
plt.plot(t, IK_FE, label="IK")
plt.plot(t, INa_FE, label="INa")
plt.plot(t, IL_FE, label="IL")
plt.title("Simulated HH Ionic Currents using the Forward Euler Approximation",fontsize = 16)
plt.xlabel("Time (ms)",fontsize = 14)
plt.ylabel("Current (nA)",fontsize = 14)
```

```
plt.legend()
plt.xlim(0,20)
plt.grid(True)
plt.show()

# Plot Gates
plt.figure(figsize=(7,4))
plt.plot(t,n_FE, label = 'n')
plt.plot(t,m_FE, label = 'm')
plt.plot(t,h_FE, label = 'h')
plt.title('Simulated HH Gating Variables using the Forward Euler Approximation',fontsize = 16)
plt.xlabel('Time(ms)',fontsize = 14)
plt.ylabel('Gating Variable',fontsize = 14)
plt.legend()
plt.xlim(0,20)
plt.grid(True)
plt.show()

#  Problem 3c: Midpoint Method
y_MP = np.array([0, n0, m0, h0], dtype=float)
dtm = 0.05
t = np.arange(0, 20, dtm)

V_MP = np.zeros_like(t)
n_MP = np.zeros_like(t)
m_MP = np.zeros_like(t)
h_MP = np.zeros_like(t)

IK_MP = np.zeros_like(t)
INa_MP = np.zeros_like(t)
IL_MP = np.zeros_like(t)

for i in range(len(t)):
    V_MP[i], n_MP[i], m_MP[i], h_MP[i] = y_MP

    IK_MP[i]  = gK_bar * n_MP[i]**4 * (V_MP[i] - EK)
    INa_MP[i] = gNa_bar * m_MP[i]**3 * h_MP[i] * (V_MP[i] - ENa)
    IL_MP[i]  = gL_bar * (V_MP[i] - EL)

    k1 = np.array(derivatives(y_MP, t[i], IStim))
```

```
    k2 = np.array(derivatives(y_MP + dtm/2 * k1, t[i] + dtm/2, IStim))
    y_MP = y_MP + dtm * k2


# Plot AP
plt.figure(figsize=(8,5))
plt.plot(t, V_MP, color='purple')
plt.title("Simulated HH AP using the Midpoint Method",fontsize = 16)
plt.xlabel("Time (ms)",fontsize = 14)
plt.ylabel("Vm (mV)",fontsize = 14)
plt.grid(True)
plt.xlim(0,20)
plt.show()

# Plot Ionic Currents
plt.figure(figsize=(7,4))
plt.plot(t, IK_MP, label="IK")
plt.plot(t, INa_MP, label="INa")
plt.plot(t, IL_MP, label="IL")
plt.title("Simulated HH Ionic Currents using the Midpoint Method",fontsize = 16)
plt.xlabel("Time (ms)",fontsize = 14)
plt.ylabel("Current (nA)",fontsize = 14)
plt.legend()
plt.grid(True)
plt.xlim(0,20)
plt.show()
# Plot Gates
plt.figure(figsize=(7,4))
plt.plot(t,n_MP, label = 'n')
plt.plot(t,m_MP, label = 'm')
plt.plot(t,h_MP, label = 'h')
plt.title('Simulated HH Gating Variables using the Midpoint Method',fontsize = 16)
plt.xlabel('Time(ms)',fontsize = 14)
plt.ylabel('Gating Variable',fontsize = 14)
plt.legend()
plt.grid(True)
plt.xlim(0,20)
plt.show()
```

Appendix D
Python Code for Problem 4

```
# Problem 4a
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
# Parameters
gNa_Norm = 70.7
gNa_bar50 = 70.7 * 0.5   # 50% reduction
gNa_bar25 = 70.7 * 0.75  # 25% reduction
gK_bar = 24.31
ENa = 110
EK = -12
Cm = 1
gL_bar = 0.3
EL = 0
t = np.linspace(0, 50, 5000)

# Rate function
def rates(Vm):
    if Vm == -10:
        a_n = 0.1
    else:
        a_n = (10 - Vm) / (100 * (np.exp((10 - Vm) / 10) - 1))
    b_n = 0.125 * np.exp(-Vm / 80)

    if Vm == -25:
        a_m = 1
    else:
        a_m = (25 - Vm) / (10 * (np.exp((25 - Vm) / 10) - 1))
    b_m = 4 * np.exp(-Vm / 18)

    a_h = 0.07 * np.exp(-Vm / 20)
    b_h = 1 / (np.exp((30 - Vm) / 10) + 1)
    return [a_n, b_n, a_m, b_m, a_h, b_h]

# Stimulus
def IStim(t):
    return 25 if 0.5 <= t <= 2.5 else 0
```

```
# Hodgkin-Huxley equations
def derivatives(y, t, gNa_bar, IStim):
    V, n, m, h = y
    a_n, b_n, a_m, b_m, a_h, b_h = rates(V)
    dndt = a_n * (1 - n) - b_n * n
    dmdt = a_m * (1 - m) - b_m * m
    dhdt = a_h * (1 - h) - b_h * h

    IK  = gK_bar  * n**4 * (V - EK)
    INa = gNa_bar * m**3 * h * (V - ENa)
    IL  = gL_bar  * (V - EL)
    I_ext = IStim(t)
    dVdt = (I_ext - (IK + INa + IL)) / Cm
    return [dVdt, dndt, dmdt, dhdt]

# Initial conditions
a_n, b_n, a_m, b_m, a_h, b_h = rates(0)
n0 = a_n / (a_n + b_n)
m0 = a_m / (a_m + b_m)
h0 = a_h / (a_h + b_h)
y0 = [0, n0, m0, h0]

# Solve for both cases
soln = odeint(derivatives, y0, t, args = (gNa_Norm, IStim))
sol50 = odeint(derivatives, y0, t, args=(gNa_bar50, IStim))
sol25 = odeint(derivatives, y0, t, args=(gNa_bar25, IStim))

V50, n50, m50, h50 = sol50.T
V25, n25, m25, h25 = sol25.T
Vn, nn, mn, hn = soln.T

#Compute currents
IK50  = gK_bar * n50**4 * (V50 - EK)
INa50 = gNa_bar50 * m50**3 * h50 * (V50 - ENa)
IL50  = gL_bar * (V50 - EL)

IK25  = gK_bar * n25**4 * (V25 - EK)
INa25 = gNa_bar25 * m25**3 * h25 * (V25 - ENa)
IL25  = gL_bar * (V25 - EL)
```

```
IKNorm = gK_bar * n25**4 * (V25 - EK)
INaNorm = gNa_Norm * m25**3 * h25 * (V25 - ENa)
ILNorm = gL_bar * (V25 - EL)

# Plot stimulus
I_values = np.array([IStim(tt) for tt in t])
plt.figure(figsize=(6,3))
plt.plot(t, I_values, color='black', lw=2)
plt.xlabel('Time (ms)', fontsize=14)
plt.ylabel('I_stim (nA)', fontsize=14)
plt.title('Stimulus Current Pulse', fontsize=16)
plt.xlim(0, 20)
plt.grid(True)
plt.show()

#Plot ionic currents
plt.figure(figsize=(8,4))
plt.plot(t, INa50, label='I_Na (50% GNa)')
plt.plot(t, INa25, '--',label='I_Na (25% GNa)')
plt.plot(t, INaNorm, label='I_Na (Normal GNa)')
plt.plot(t, IK50, label='I_K (50% GNa)')
plt.plot(t, IK25,'--', label='I_K (25% GNa)')
plt.plot(t, IKNorm, label='I_K (Normal GNa)')
plt.plot(t, IL50, label='I_L (50% GNa)')
plt.plot(t, IL25,'--', label='I_L (25% GNa)')
plt.plot(t, ILNorm, label='I_L (Normal GNa)')
plt.title('Ionic Currents with 25% and 50% Reduction of GNa', fontsize=16)
plt.xlabel('Time (ms)', fontsize=14)
plt.ylabel('Ionic Currents (nA)', fontsize=14)
plt.legend(loc = 'upper right')
plt.xlim(0, 20)
plt.grid(True)
plt.show()

# Plot action potentials
plt.figure(figsize=(8,5))
plt.plot(t, Vn, label='Normal GNa', color='purple', lw=2)
plt.plot(t, V50, label='50% GNa', color='orange', lw=2)
plt.plot(t, V25, label='25% GNa', color='green', lw=2)
plt.title('Action Potentials with 25% and 50% Reduction of GNa', fontsize=16)
```

```
plt.xlabel('Time (ms)', fontsize=14)
plt.ylabel('Membrane Voltage (mV)', fontsize=14)
plt.legend()
plt.xlim(0, 20)
plt.grid(True)
plt.show()

# Plot gating variables
plt.figure(figsize=(8,4))
plt.plot(t, n50, label='n (50%)')
plt.plot(t, n25, '--',label='n (25%)')
plt.plot(t, nn, label='n (Normal)')
plt.plot(t, m50, label='m (50%)')
plt.plot(t, m25,'--', label='m (25%)')
plt.plot(t, mn, label='m (Normal)')
plt.plot(t, h50,  label='h (50%)')
plt.plot(t, h25, '--', label='h (25%)')
plt.plot(t, hn,label='h (Normal)')
plt.title('Gating Variables for 25% and 50% GNa Reduction', fontsize=16)
plt.xlabel('Time (ms)', fontsize=14)
plt.ylabel('Gating Variable', fontsize=14)
plt.legend(loc = 'upper right')
plt.xlim(0, 20)
plt.grid(True)
plt.show()
```

 # Problem 4b

```
# Function to Create two pulses of the same magnitude a given delta t apart
def IStim_double(t, dt, amp=25):
    if (0.5 <= t <= 2.5) or (0.5 + dt <= t <= 2.5 + dt):
        return amp
    else:
        return 0

# Function to stimulate and measure excitability
def simulate_dt(dt, amp):
    # Use longer time to ensure we capture both pulses
    t = np.linspace(0, 50 + dt, 6000)
    Is_func = lambda tt: IStim_double(tt, dt, amp)
```

```
    # Use fresh initial conditions for each simulation
    a_n, b_n, a_m, b_m, a_h, b_h = rates(0)
    n0 = a_n / (a_n + b_n)
    m0 = a_m / (a_m + b_m)
    h0 = a_h / (a_h + b_h)
    V0 = 0
    y0_local = [V0, n0, m0, h0]  # Local initial conditions

    sol = odeint(derivatives, y0_local, t, args=(gNa_bar,Is_func))
    V = sol[:, 0]

    # Count spikes using simple threshold detection
    spike_inds = np.where((V[1:] > 0) & (V[:-1] <= 0))[0]
    num_spikes = len(spike_inds)
    return num_spikes, V, t

# Reset gNa_bar to reduced value for the first part (50%)
gNa_bar = gNa_Norm * 0.5

dt_values = np.arange(1, 31, 1)  # delta t values
I1 = 25
I2_ratio_reduced = []

for dt in dt_values:
    amp = I1
    spikes, _, _ = simulate_dt(dt, amp)

    if spikes < 2:
        amp_test = I1
        while amp_test < 600:  # upper cap
            spikes, _, _ = simulate_dt(dt, amp_test)
            if spikes >= 2:
                break
            amp_test += 2
        I2_ratio_reduced.append(amp_test / I1)
    else:
        I2_ratio_reduced.append(1.0)

I_ratio_reduced = np.array(I2_ratio_reduced)
```

```
# Rerun for 25% GNa
gNa_bar = gNa_Norm * 0.75

I1 = 25
I2_ratio_25 = []

for dt in dt_values:
    amp = I1
    spikes, _, _ = simulate_dt(dt, amp)

    if spikes < 2:
        amp_test = I1
        while amp_test < 600:  # upper cap
            spikes, _, _ = simulate_dt(dt, amp_test)
            if spikes >= 2:
                break
            amp_test += 2
        I2_ratio_25.append(amp_test / I1)
    else:
        I2_ratio_25.append(1.0)

I_ratio_25 = np.array(I2_ratio_25)
# Rerun for normal Gna
gNa_bar = gNa_Norm  # Reset to normal value
I2_ratio_norm = []

for dt in dt_values:
    amp = I1
    spikes, _, _ = simulate_dt(dt, amp)

    if spikes < 2:
        amp_test = I1
        while amp_test <600:
            spikes, _, _ = simulate_dt(dt, amp_test)
            if spikes >= 2:
                break
            amp_test += 2
        I2_ratio_norm.append(amp_test / I1)
    else:
```

```
        I2_ratio_norm.append(1.0)

I_ratio_norm = np.array(I2_ratio_norm)

# Plot Refractory Period Curve
plt.figure(figsize=(7,5))
plt.plot(dt_values, I_ratio_norm, label='Normal $g_{Na}$', color='blue', lw=2)
plt.plot(dt_values, I_ratio_reduced, label='50% $g_{Na}$', color='red', lw=2)
plt.plot(dt_values, I_ratio_25, label='25% $g_{Na}$', color='green', lw=2)
plt.axhline(y=1, color='k', linestyle='--', linewidth=1)
plt.xlim(0, 30)
plt.ylim(0, 20)
plt.xlabel(r'$\Delta t$ (ms)', fontsize=14)
plt.ylabel(r'$\frac{I_2}{I_1}$', fontsize=14)
plt.title('Refractory Period Comparison: Normal vs 50% $g_{Na}$ vs 25% $g_{Na}$ ',
fontsize=16)
plt.legend(fontsize=12)
plt.grid(True)
plt.show()
```

 # Problem 4d

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint


# Parameters
gNa_bar = 70.7
gK_Norm = 24.31
gK_bar50 = gK_Norm * 0.5
gK_bar25 = gK_Norm * 0.75
ENa = 110
EK = -12
Cm = 1
gL_bar = 0.3
EL = 0
t = np.linspace(0, 50, 5000)


# Rate Functions
def rates(Vm):
```

```
    if Vm == -10:
        a_n = 0.1
    else:
        a_n = (10 - Vm) / (100 * (np.exp((10 - Vm) / 10) - 1))
    b_n = 0.125 * np.exp(-Vm / 80)

    if Vm == -25:
        a_m = 1
    else:
        a_m = (25 - Vm) / (10 * (np.exp((25 - Vm) / 10) - 1))
    b_m = 4 * np.exp(-Vm / 18)

    a_h = 0.07 * np.exp(-Vm / 20)
    b_h = 1 / (np.exp((30 - Vm) / 10) + 1)
    return [a_n, b_n, a_m, b_m, a_h, b_h]


# Stimulus
def IStim(t):
    return 10 if 0.5 <= t <= 2.5 else 0

# HH Equations
def derivatives(y, t, gK_bar, IStim):
    V, n, m, h = y
    a_n, b_n, a_m, b_m, a_h, b_h = rates(V)
    dndt = a_n * (1 - n) - b_n * n
    dmdt = a_m * (1 - m) - b_m * m
    dhdt = a_h * (1 - h) - b_h * h

    IK  = gK_bar  * n**4 * (V - EK)
    INa = gNa_bar * m**3 * h * (V - ENa)
    IL  = gL_bar  * (V - EL)

    I_ext = IStim(t)
    dVdt = (I_ext - (IK + INa + IL)) / Cm
    return [dVdt, dndt, dmdt, dhdt]


# Initial Conditions
a_n, b_n, a_m, b_m, a_h, b_h = rates(0)
```

```
n0 = a_n / (a_n + b_n)
m0 = a_m / (a_m + b_m)
h0 = a_h / (a_h + b_h)
y0 = [0, n0, m0, h0]


# Solve for Normal, 50%, and 25% gK
sol_norm = odeint(derivatives, y0, t, args=(gK_Norm, IStim))
sol50 = odeint(derivatives, y0, t, args=(gK_bar50, IStim))
sol25 = odeint(derivatives, y0, t, args=(gK_bar25, IStim))

Vn, nn, mn, hn = sol_norm.T
V50, n50, m50, h50 = sol50.T
V25, n25, m25, h25 = sol25.T


# Compute Ionic Currents
def compute_currents(V, n, m, h, gK_bar):
    IK  = gK_bar  * n**4 * (V - EK)
    INa = gNa_bar * m**3 * h * (V - ENa)
    IL  = gL_bar  * (V - EL)
    return IK, INa, IL

IKn, INan, ILn = compute_currents(Vn, nn, mn, hn, gK_Norm)
IK50, INa50, IL50 = compute_currents(V50, n50, m50, h50, gK_bar50)
IK25, INa25, IL25 = compute_currents(V25, n25, m25, h25, gK_bar25)


# Plot: Stimulus
I_values = np.array([IStim(tt) for tt in t])
plt.figure(figsize=(6,3))
plt.plot(t, I_values, color='black', lw=2)
plt.xlabel('Time (ms)', fontsize=14)
plt.ylabel('I_stim (nA)', fontsize=14)
plt.title('Stimulus Current Pulse', fontsize=16)
plt.xlim(0, 20)
plt.grid(True)
plt.show()

# Plot: Ionic Currents
```

```
plt.figure(figsize=(8,4))
plt.plot(t, IKn, label='I_K (Normal)')
plt.plot(t, IK50, label='I_K (50%)')
plt.plot(t, IK25,'--',  label='I_K (25%)')
plt.plot(t, INa25,'--', label='I_Na (25%)')
plt.plot(t, INa50, label='I_Na (50%)')
plt.plot(t,INan, label = 'I_Na (Normal)')
plt.plot(t, IL25, '--', label='I_L (25%)')
plt.plot(t, IL50, label='I_L (50%)')
plt.plot(t, ILn, label='I_L (Normal)')
plt.title('Ionic Currents with Reduced gK', fontsize=16)
plt.xlabel('Time (ms)', fontsize=14)
plt.ylabel('Current (nA)', fontsize=14)
plt.legend(loc = 'upper right')
plt.xlim(0, 20)
plt.grid(True)
plt.show()


# Plot: Action Potentials
plt.figure(figsize=(8,5))
plt.plot(t, Vn, label='Normal gK', color='blue', lw=2)
plt.plot(t, V50, label='50% gK', color='red', lw=2)
plt.plot(t, V25, label='25% gK', color='green', lw=2)
plt.title('Action Potentials with gK Reduction', fontsize=16)
plt.xlabel('Time (ms)', fontsize=14)
plt.ylabel('Membrane Voltage (mV)', fontsize=14)
plt.legend()
plt.xlim(0, 20)
plt.grid(True)
plt.show()


# Plot: Gating Variables
plt.figure(figsize=(8,4))
plt.plot(t, n50, label='n (50%)')
plt.plot(t, n25, '--', label='n (25%)')
plt.plot(t,nn, label = 'n (Normal)')
plt.plot(t, m50, label='m (50%)')
plt.plot(t, m25, '--', label='m (25%)')
```

```
plt.plot(t,mn, label = 'm (Normal)')
plt.plot(t, h50, label='h (50%)')
plt.plot(t, h25, '--', label='h (25%)')
plt.plot(t,hn, label = 'h (Normal)')
plt.title('Gating Variables for gK Reduction', fontsize=16)
plt.xlabel('Time (ms)', fontsize=14)
plt.ylabel('Gating Variable', fontsize=14)
plt.legend()
plt.xlim(0, 20)
plt.grid(True)
plt.show()


# Refractory Period Analysis
def IStim_double(t, dt, amp=10):
    if (0.5 <= t <= 2.5) or (0.5 + dt <= t <= 2.5 + dt):
        return amp
    else:
        return 0

def simulate_dt(dt, amp, gK_bar):
    t = np.linspace(0, 50 + dt, 6000)
    Is_func = lambda tt: IStim_double(tt, dt, amp)
    sol = odeint(derivatives, y0, t, args=(gK_bar, Is_func))
    V = sol[:, 0]
    spike_inds = np.where((V[1:] > 0) & (V[:-1] <= 0))[0]
    num_spikes = len(spike_inds)
    return num_spikes, V, t

dt_values = np.arange(1, 31, 1)
I1 = 10

#  50% gK
gK_bar = gK_bar50
I2_ratio_reduced = []
for dt in dt_values:
    amp = I1
    spikes, _, _ = simulate_dt(dt, amp, gK_bar)
    if spikes < 2:
        amp_test = I1
```

```
        while amp_test < 400:
            spikes, _, _ = simulate_dt(dt, amp_test, gK_bar)
            if spikes >= 2:
                break
            amp_test += 2
        I2_ratio_reduced.append(amp_test / I1)
    else:
        I2_ratio_reduced.append(1.0)

#  25% gK
gK_bar = gK_bar25
I2_ratio_25 = []
for dt in dt_values:
    amp = I1
    spikes, _, _ = simulate_dt(dt, amp, gK_bar)
    if spikes < 2:
        amp_test = I1
        while amp_test < 400:
            spikes, _, _ = simulate_dt(dt, amp_test, gK_bar)
            if spikes >= 2:
                break
            amp_test += 2
        I2_ratio_25.append(amp_test / I1)
    else:
        I2_ratio_25.append(1.0)

# Normal gK
gK_bar = gK_Norm
I2_ratio_norm = []
for dt in dt_values:
    amp = I1
    spikes, _, _ = simulate_dt(dt, amp, gK_bar)
    if spikes < 2:
        amp_test = I1
        while amp_test < 400:
            spikes, _, _ = simulate_dt(dt, amp_test, gK_bar)
            if spikes >= 2:
                break
            amp_test += 2
        I2_ratio_norm.append(amp_test / I1)
```

```
    else:
        I2_ratio_norm.append(1.0)


# Plot: Refractory Period Curves
plt.figure(figsize=(7,5))
plt.plot(dt_values, I2_ratio_norm, label='Normal $g_K$', color='blue', lw=2)
plt.plot(dt_values, I2_ratio_reduced, label='50% $g_K$', color='red', lw=2)
plt.plot(dt_values, I2_ratio_25, label='25% $g_K$', color='green', lw=2)
plt.axhline(y=1, color='k', linestyle='--', linewidth=1)
plt.xlim(0, 30)
plt.ylim(0, 20)
plt.xlabel(r'$\Delta t$ (ms)', fontsize=14)
plt.ylabel(r'$\frac{I_2}{I_1}$', fontsize=14)
plt.title('Refractory Period Comparison: Normal vs Reduced $g_K$', fontsize=16)
plt.legend(fontsize=12)
plt.grid(True)
plt.show()
```